计算机体系结构

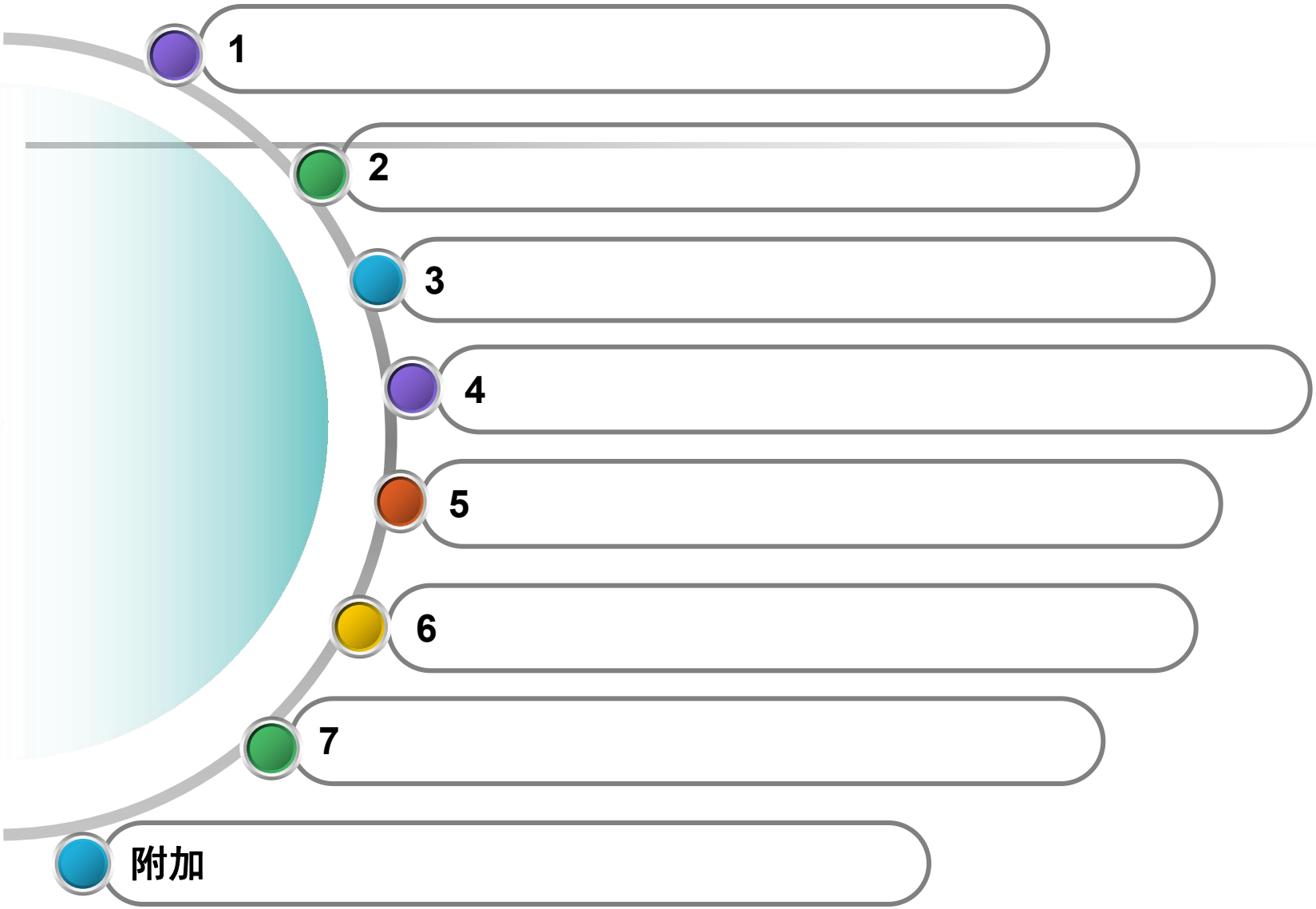
主讲：何世文

2025年6月4日

Email: shiwen.he.hn@csu.edu.cn

中南大学计算机学院

习题内容



第一章 计算机系统结构简介

系统结构的相关概念

- 计算机系统的层次结构概念
- 广义机器、虚拟机器、透明性、编译、解释
- 计算机系统结构、组织和实现
- 计算机系统分类方法\ Flynn 分类法

系统分析技术

- 大概率事件优先原理
- Amdahl定律、加速比定义
- 程序访存的局部性原理
- CPU性能公式

$$S_e = \frac{T_0}{T_n} = \frac{1}{1 - F_e + F_e / S_e}$$

性能评价标准

- 性能指标（CPU时间， CPI， MIPS,MFLOPS)
- 性能比较

$$\mathrm{CPI} = \sum_{i=1}^{n} (CPI_i \times I_i) / I_N = \sum_{i=1}^{n} (CPI_i \times I_i / I_N)$$

第一章

Amdahl定律：$$S_n = \frac{T_0}{T_n} = \frac{1}{(1-Fe) + \frac{Fe}{Se}}$$

CPU时间 = IC ×CPI×时钟周期时间

$$= \sum_{i=1}^{n} (CPI_i \times IC_i) \times \text{时钟周期时间}$$

$$CPI = \frac{\text{时钟周期数}}{IC} = \frac{\sum_{i=1}^{n} (CPI_i \times IC_i)}{IC} = \sum_{i=1}^{n} (CPI_i \times \frac{IC_i}{IC})$$

$$MIPS \text{ 速率} = \frac{f}{CPI \times 10^6}$$

习题1.7

对于一台400MHz计算机执行标准测试程序，程序中指令类型，执行数量和平均时钟周期数如下：

指令类型	指令执行数量	平均时钟周期数
整数	45000	1
数据传送	75000	2
浮点	8000	4
分支	1500	2

求该计算机的有效CPI、MIPS和程序执行时间。

解：

$$CPI = \sum_{i=1}^{4}(CPI_i \times \frac{IC_i}{IC}) = 1\times\frac{45000}{129500} + 2\times\frac{75000}{129500} + 4\times\frac{8000}{129500} + 2\times\frac{1500}{129500} = \frac{230000}{129500} \approx 1.776$$

$$MIPS = \frac{f}{CPI\times 10^6} \approx \frac{400\times 10^6}{1.776\times 10^6} \approx \frac{400}{1.776} \approx 225.225 MIPS$$

$$T = \frac{IC}{MIPS\times 10^6} \approx \frac{129500\times 1.776}{400\times 10^6} \approx 0.000575(\text{秒}) = 0.575(\text{毫秒})$$

习题1.10

计算机系统有三个部件可以改进，这三个部件的加速比如下：部件加速比1＝30；部件加速比2＝20；部件加速比3＝10；

（1）如果部件1和部件2的可改进比例为30%，那么当部件3的可改进比例为多少时，系统的加速比才可以达到10？

（2）如果三个部件的可改进比例为30%、30%和20%，三个部件同时改进，那么系统中不可加速部分的执行时间在总执行时间中占的比例是多少？

$$T_e = T_o\left[(1-f_e)+\frac{f_e}{S_e}\right] \qquad S=\frac{1}{(1-f_e)+\frac{f_e}{S_e}} \qquad S=\frac{1}{(1-\sum_i f_i)+\sum_i \frac{f_i}{S_i}}$$

习题1.10

$$S=\left\{[1-(f_1+f_2+f_3)]+\frac{f_1}{S_1}+\frac{f_2}{S_2}+\frac{f_3}{S_3}\right\}^{-1}$$

$$10=\left\{[1-(0.3+0.3+f_3)]+\frac{0.3}{30}+\frac{0.3}{20}+\frac{f_3}{10}\right\}^{-1} \qquad f_3=\frac{65}{180}=0.36$$

$$p=\frac{[1-(0.3+0.3+0.2)]T}{\frac{0.3T}{30}+\frac{0.3T}{20}+\frac{0.2T}{10}+0.2T}$$

$$=\frac{0.2}{\frac{0.3}{30}+\frac{0.3}{20}+\frac{0.2}{10}+0.2}$$

$$=\frac{0.2}{\frac{0.6}{60}+\frac{0.9}{60}+\frac{1.2}{60}+\frac{12}{60}}$$

$$=\frac{12}{14.7}=0.82$$

习题1.11

given

假设浮点数指令FP指令的比例为30%，其中浮点数平方根FPSQR占全部指令的比例为4%，FP操作的CPI为5，FPSQR操作的CPI为20，其他指令的平均CPI为1.25。

现有两种改进方案，

第一种：把FPSQR操作的CPI减至3

第二种：把所有的FP操作的CPI减至3

试比较两种方案对系统性能的提高程度。

解法1：

利用原始CPI的唯一性，先使用已知条件求出原始CPI，再求出除去FPSQR指令外其他指令的平均CPI，最后比较改进后的CPI大小。

原始CPI = 5 × 30% + 1.25 × (1 - 30%) = 2.375

设除FPSQR外其余指令的平均CPI为X

则 2.375 = 20 × 4% + (1 - 4%)X ，解出X = 1.640625

方案1：CPI_1 = 3 × 4% + 1.640625 × (1 - 4%) = 1.695

方案2：CPI_2 = 3 × 30% + 1.25 × (1 - 30%) = 1.775

结论： 方案1导致的新CPI更小，性能更好

习题1.11

解法2：

用Amdahl公式求。记指令总条数=M，时钟周期长度=CYCLE。

原始总时间T_{old} = 0.3M × 5 × CYCLE + 0.7M × 1.25 × CYCLE

= M × 2.375 × CYCLE

T_{FP} = 0.3M × 5 × CYCLE = M × 1.5 × CYCLE，

所占比例为1.5/2.375 ≈ 63%

T_{FPSQR} = 0.04M × 20 × CYCLE = M × 0.8 × CYCLE，

所占比例为0.8/2.375 ≈ 34%

方案1：Se = 20/3，Fe ≈ 34%，Sn_1 = 1 / [(1 - Fe) + Fe / Se] ≈ 1.4

方案2：Se = 5/3，Fe ≈ 63%，Sn_2 = 1 / [(1 - Fe) + Fe / Se] ≈ 1.3

结论：　方案1导致加速比更大，性能更好

2

假设在一台40MHz处理机上运行200,000条指令的目标代码，程序主要由四种指令组成。根据程序跟踪实验结果，已知指令混合比和每种指令所需的指令数如下：

指令类型	CPI	指令混合比
算术和逻辑	1	60%
高速缓存命中的加载/存储	2	18%
转移	4	12%
高速存储缺失的存储器访问	8	10%

(a)计算在单处理机上用上述跟踪数据运行程序的平均CPI。

(b)根据(a)所得的CPI,计算相应的MIPS速率。

(a) $CPI = 1*60\% + 2*18\% + 4*12\% + 8*10\%$

=2.24

(b) $$MIPS = \frac{f}{CPI * 10^6} = \frac{40MHZ}{2.24 * 10^6} = 17.86$$

第二章 指令系统

- **ISA 核心概念**
- 指令的表示
- 指令格式优化
- 指令系统优化
- **CISC、RISC**

- **CISC（复杂指令集）**：如 x86，指令功能复杂，通过微程序实现，代表机型有 Intel 80x86。
- **RISC（精简指令集）**：如 MIPS、RISC-V，指令简单规整，硬布线控制，强调流水线效率。

存储器寻址

1. **编址单位**：几乎所有现代机器按字节编址，字长定义因架构而异（如 MIPS 字长 32 位，x86 字长 16/32/64 位）。
2. **端序（Endianness）**
   1. **大端（Big Endian）**：高字节存低地址，如 IBM 360、MIPS。
   2. **小端（Little Endian）**：低字节存低地址，如 Intel 80x86、DEC VAX。
3. **对齐要求**：数据对象地址需为其大小的整数倍，避免跨存储体访问降低效率。

1.指令格式

- **R 型（寄存器 - 寄存器）**：如 MIPS 的add rd, rs1, rs2，操作码 + 寄存器号 + 功能码。
- **I 型（立即数 /load/store）**：如addi rd, rs1, imm，操作码 + 寄存器号 + 立即数。
- **B 型（分支）**：如beq rs1, rs2, offset，操作码 + 寄存器号 + 偏移量。
- **U 型（长立即数）**：如lui rd, imm，操作码 + 立即数高 20 位。
- **J 型（跳转）**：如jal rd, offset，操作码 + 偏移量。

2.寻址方式

- **寄存器寻址**：操作数在寄存器中，如add r1, r2, r3。
- **立即数寻址**：操作数在指令中，如addi r1, r2, 10。
- **变址寻址**：EA = 基址寄存器 + 偏移量，如lw r1, 4(r2)。
- **相对寻址**：PC + 偏移量，如分支指令beq r1, r2, Label。

3. 控制流

- **条件分支：如beq（相等分支）、bne（不等分支），基于寄存器比较结果跳转。**
- **无条件跳转：如j Label、jal（跳转并链接），用于函数调用。**
- **过程调用与返回：**
  - **调用**：jal ra, func（保存返回地址到ra并跳转）。
  - **返回**：jr ra或ret（从ra取地址跳转）。
- **异常与中断：通过陷阱指令（如ecall）触发，需保存现场并切换至异常处理程序。**

4.指令编码与优化

1.操作码编码

- **固定长度**：如 MIPS 所有指令 32 位，便于流水线译码。
- **可变长度**：如 x86，操作码长度 1-15 字节，需复杂译码逻辑。

2.霍夫曼编码：按指令使用频度分配码长，高频指令短码，降低平均码长。

3.扩展编码：如 RISC-V 的 32 位指令结合 16 位压缩指令（C 扩展），减少代码体积。

ISA 性能优化

（一）减少 CPI（时钟周期数）

1.流水线优化：RISC 架构通过简单指令实现单周期执行，如 MIPS 五级流水线（取指 - 译码 - 执行 - 访存 - 写回）。

2.分支预测：预测分支走向，减少流水线气泡，如动态分支预测器。

3.寄存器重命名：消除数据相关，支持乱序执行，如 x86 的超标量实现。

（二）减少指令数（IC）

1.高级语言支持：ISA 提供对高级语言特性的直接支持（如循环、数组）。

2.指令融合：合并多条指令为一条，如 x86 的 SSE 指令合并多个浮点操作。

（三）降低失效率

1.Cache 友好设计：支持对齐访存、批量数据传输（如 MIPS 的lw/sw）。

2.预取指令：硬件 / 软件预取后续数据，减少访存延迟。

特性	RISC	CISC
指令数量	少（约 40-100 条）	多（数百至上千条）
指令长度	固定（如 32 位）	可变（1-15 字节）
指令周期	单周期执行	多周期执行（微程序控制）
寄存器数量	多（32 个通用寄存器）	少（8-16 个，部分专用）
访存模式	仅 load/store 指令访存	支持存储器 - 存储器操作
控制逻辑	硬布线控制，流水线效率高	微程序控制，灵活性高但速度慢
编译优化	依赖编译器生成高效代码	编译器负担轻，指令直接对应高级语言

（一）指令格式设计案例

1.问题：设计支持 32 个寄存器、16 种操作、3 种寻址方式的指令格式。

2.方案

- **操作码**：4 位（16 种操作）。
- **寻址方式**：2 位（寄存器 / 立即数 / 变址）。
- **寄存器号**：5 位（32 个寄存器）。
- **立即数**：16 位（支持 - 32768~32767）。
- **格式组合**：
  - R 型：opcode(4) + addr_mode(2) + rs(5) + rt(5) + rd(5)。
  - I 型：opcode(4) + addr_mode(2) + rs(5) + rt(5) + imm(16)。

（二）性能计算实例

1.场景：某程序执行 10^6 条指令，CPI=2，时钟频率 3GHz。

3.计算

- **执行时间**：(10^6 × 2) / 3×10^9 = 0.000667秒 = 667μs。
- **MIPS**：10^6 / 0.000667 ≈ 1500 MIPS。

- **……**

- 一台模型机共有7条指令，各指令的使用频度分别为35%，25%，20%，10%，5%，3%，2%，有8个通用数据寄存器，2个变址寄存器。
  - (1)要求操作码的平均长度最短，请设计操作码的编码，并计算所设计操作码的平均长度。
  - (2)设计8位字长的寄存器-寄存器型指令3条，16位字长的寄存器-存储器型变址寻址方式指令4条，变址范围不小于正、负127。请设计指令格式，并给出各字段的长度和操作码的编码。

指令系统

(1)

指令序号	出现的概率	编码法
I_1	0.35	00
I_2	0.25	01
I_3	0.20	10
I_4	0.10	110
I_5	0.05	1110
I_6	0.03	11110
I_7	0.02	11111
操作码的平均长度		2.35

平均长度H＝0.35*2+0.25*2+0.20*2+0.10*3+0.05*4+0.03*5+0.02*5=2.35

指令系统

- (2) 寄存器-寄存器型指令格式

指令序号	编码（2位）	通用寄存器（3位）	通用寄存器（3位）
I1	00	R	R
I2	01	R	R
I3	10	R	R

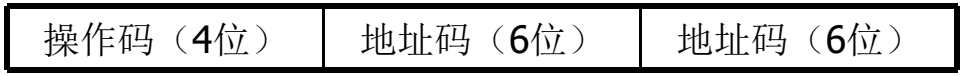
- 寄存器-存储器型变址寻址方式指令格式

指令序号	编码（4位）	通用寄存器（3位）	变址寄存器（1位）	地址偏移量（8位）
I4	1100	R	X	A
I5	1101	R	X	A
I6	1110	R	X	A
I7	1111	R	X	A

- 某处理机的指令字长为16位，有双地址指令、单地址指令和零地址指令三类，并假设每个地址字段的长度均为6位。
  - 如果双地址指令有15条，单地址指令和零地址指令的条数基本相同，问单地址指令和零地址指令各有多少条？并且为这三类指令分配操作码。
  - 如果要求三类指令的比例大致为1：9：9，问双地址指令、单地址指令和零地址指令各有多少条？并且为这三类指令分配操作码。

指令系统

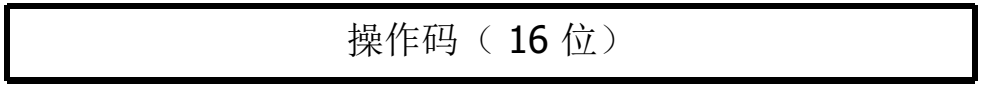
- 分析：
  - 双地址



    - 操作码4位，共2^4=16种操作码， 15条双地址指令占用了15个码点，剩2^4-15个码点作为扩展标志。
  - 单地址



    - 操作码10位，每个剩余码点扩展出6位操作码，所以，最多可以表示单地址指令$(2^4-15)2^6$条，即2^6条。
  - 零地址



    - 操作码16位，每个剩余码点扩展出6位操作码，若设单地址指令为X条，则最多可以表示零地址指令（2^6－X）2^6条。

指令系统

- （1）设单地址指令有X条，则剩余2^6—X个码点用作零地址的扩展标志，零地址指令数量可以表示为（2^6—X）2^6，据题意有

  （2^6—X）2^6 ＝X，解得X　64

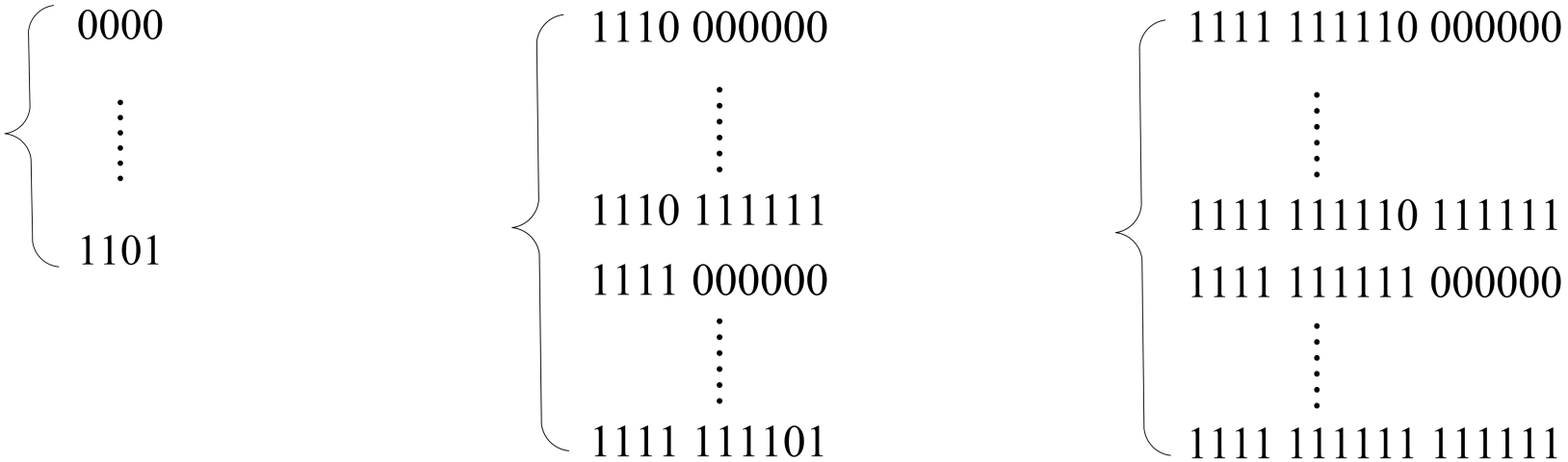
- 因为，单地址的操作码要留出一个码点用作零地址的扩展，故X＝63，单地址指令有63条，零地址指令有64条。

指令系统

- （2）由题意知，双地址指令预留给单地址的码点与单地址指令预留给零地址的码点相同。
- 设双地址指令X条，则$2^4-X=(2^4-X)2^4-9X$，. X=14

[(2的4次方 - x) 2的6次方 - 9x] 2的6次方 = 9x

- 双地址（14条）　　单地址（$2^7-2=126$条 ）零地址（2^7条 ）



第三章 流水线

流水线技术

流水线的概念： 静（动）态流水线、单（多）功能流水线

线性流水线、非线性流水线

流水线表示方法--时空图 、连接图

流水线特点 ： 五段流水线（访存部件ME, 转移EX)

冒险与处理

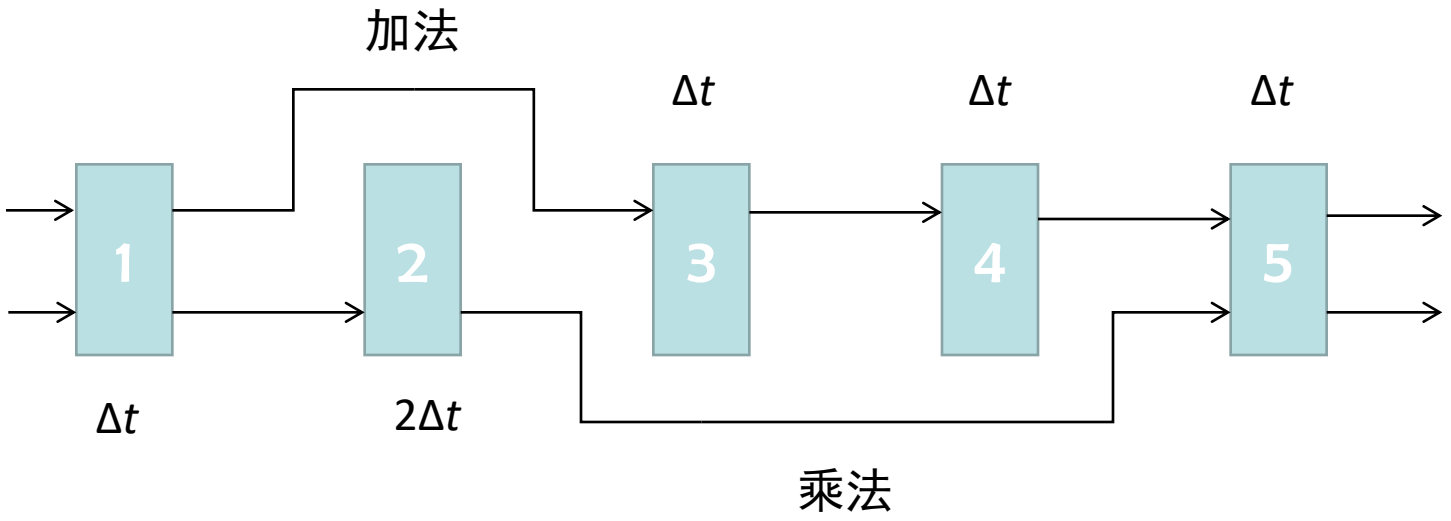
瓶颈段处理

性能指标 ： 吞吐率、加速比、效率、 最佳段数

指令 / 流水段	ALU	LOAD/STORE	BRANCH
IF	取指	取指	取指
ID	译码，读寄存器堆	译码，读寄存器堆	译码，读寄存器堆
EX	执行	计算访存有效地址	计算转移目标地址，设置条件码
ME	(空操作)	访问存储器(读或写)	若条件成立，将转移目标地址送 PC
WB	结果写回寄存器堆	将读出的数据写入寄存器堆	(空操作)

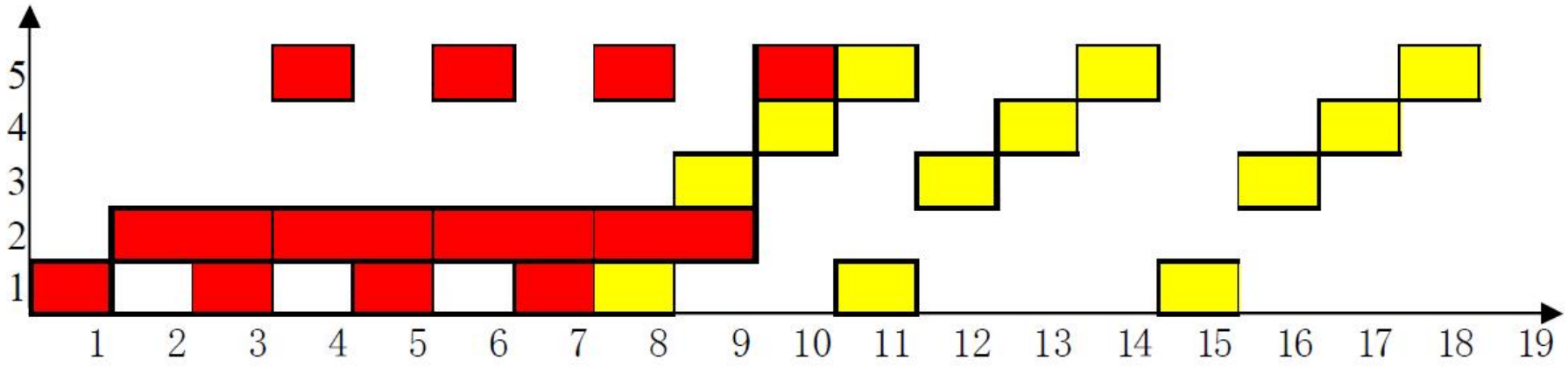
习题3.8（时空图，性能指标）

3.8有一条动态多功能流水线由5段组成，加法用1、3、4、5段，乘法用1、2、5 段，第2 段的时间为2△t，其余各段的时间均为△t，而且流水线的输出可以直接返回输入端或暂存于相应的流水寄存器中。现要在该流水线上计算 $\sum_{i=1}^{4}(A_i \times B_i)$ ，画出其时空图，并计算其吞吐率、加速比和效率。



习题3.8

首先，应选择适合于流水线工作的算法。对于本题，应先计算A1×B1、A2×B2、A3×B3和A4×B4；再计算(A1×B1) ＋ (A2×B2)和(A3×B3) ＋ (A4×B4)；然后求总的结果。



习题3.8

由图可见，它在18个△t 时间中，给出了7 个结果。所以吞吐率为：

$$TP = \frac{7}{18\Delta t}$$

如果不用流水线，由于一次求积需3△t，一次求和需4△t，则产生上述7个结果共需（4×4+3×4）△t =28△t。所以加速比为：

$$S = \frac{28\Delta t}{18\Delta t} \approx 1.56$$

该流水线的效率可由阴影区的面积和5 个段总时空区的面积的比值求得：

$$E = \frac{4\times 4 + 3\times 4}{5\times 18} \approx 0.31$$

一条线性静态多功能流水线由6个功能段组成，加法操作使用其中的1、2、3、6功能段，乘法操作使用其中的1、4、5、6功能段，每个功能段的延迟时间均相等。流水线的输出端与输入端之间有直接数据通路，而且设置有足够的缓冲寄存器。用这条流水线计算$F=\sum_{i=0}(Ai \times Bi)$，画出流水线时空图，计算流水线的实际吞吐率、加速比和效率。

标量处理机

- **Answer:**
  - 所执行的指令为：

I1: A1*B1->R1

I2: A2*B2->R2

I3: A3*B3->R3

I4: A4*B4->R4

I5: A5*B5->R5

I6: A6*B6->R6

I7: R1+R2->R7

I8: R3+R4->R8

I9: R5+R6->R9

I10: R7+R8->R10

I11: R9+R10->R11

S6				1	2	3	4	5	6				7	8	9			10				11
S5			1	2	3	4	5	6														
S4		1	2	3	4	5	6															
S3												7	8	9			10				11	
S2											7	8	9			10				11		
S1	1	2	3	4	5	6				7	8	9			10				11			
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22

吞吐率为：Tp=11/22t=1/2t

加速比:　**S**=11*4t/22t=2

效率：E=(11*4t)/(6*22t)=1/3

3.9 列举出下面循环中的所有相关，包括输出相关、反相关、真相关。

```
for (i=2; i<100; i=i+1)
    a[i]=b[i]+a[i]          ;/* s1 */
    c[i+1]=a[i]+d[i]        ; /* s2 */
    a[i-1]=2*b[i]           ; /* s3 */
    b[i+1]=2*b[i]           ;/* s4 */
```

解：展开循环两次：

```
a[i] = b[i] + a[i]          ; /* s1 */
c[i+1] = a[i] + d[i]        ; /* s2 */
a[i-1] = 2 * b[i]           ; /* s3 */
b[i+1] = 2 * b[i]           ; /* s4 */
a[i+1] = b[i+1] + a[i+1]    ; /* s1' */
c[i+2] = a[i+1] + d[i+1]    ; /* s2 '*/
a[i] = 2 * b[i+1]           ; /* s3 '*/
b[i+2] = 2 * b[i+1]         ; /* s4 '*/
```

输出相关：无

反相关：无

真相关：S1&S2

由于循环引入的相关：S4&S4'（真相关）、S1'&S4（真相关）、S3'&S4（真相关）、S1&S3'（输出相关、反相关）、S2&S3'（反相关）。

习题3.10（单功能非线性流水线调度）

有一个5段流水线，各段执行时间均为Δt，其预约表如下：

时间 功能段	1	2	3	4	5	6	7
S1	√						√
S2		√			√		
S3			√	√			
S4				√			√
S5					√	√	

（1）画出流水线任务调度的状态转移图。

（2）分别求出允许不等时间间隔调度和等时间间隔调度的两种最优调度策略，以及这两种调度策略的流水线最大吞吐率。

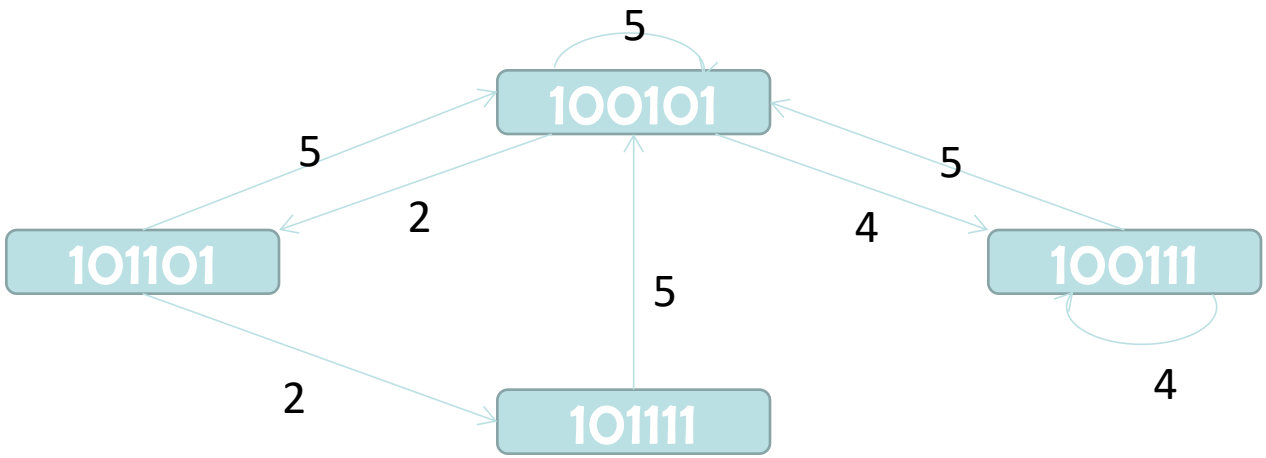
（3）若连续输入10个任务，求这两种调度策略的流水线实际吞吐率和加速比。

习题3.10

（1）由预约表可得禁止表：F={6, 3, 1}；

根据禁止表得到初始冲突向量：C0=（100101）；

流水线任务调度的状态转移图为：



习题3.10

（2）由状态转移图可得不发生段争用冲突的调度策略以及平均延迟时间如下所示。

调度策略	平均延迟时间		调度策略	平均延迟时间
(2,2,5)	3Δt		(4,5)	4.5Δt
(2,5)	3.5Δt		(5)	5Δt
(4) 间隔	4Δt			

由上可知，允许不等时间间隔调度的最优调度策略是(2, 2, 5)，流水线最大吞吐率为： $1/3\Delta t$。

等时间间隔的调度的最优调度策略是(4)，流水线最大吞吐率为：$1/4\Delta t$。

习题3.10

(3)按调度策略(2,2,5)，连续输入10 个任务的流水线实际吞吐率与加速比分别为：

$$TP_1 = \frac{10}{(2+2+5+2+2+5+2+2+5+7)\Delta t} = \frac{10}{34\Delta t}$$

$$S_1 = \frac{10 \times 7\Delta t}{34\Delta t} \approx 2.06$$

按调度策略(4)，连续输入10 个任务的流水线实际吞吐率与加速比分别为：

$$TP_2 = \frac{10}{(4 \times 9 + 7)\Delta t} = \frac{10}{43\Delta t}$$

$$S_2 = \frac{10 \times 7\Delta t}{43\Delta t} \approx 1.63$$

习题3.11（相关，定向，指令调度）

在MIPS流水线(按照图3.33)上运行如下代码序列：

```
LOOP:   LW      R1, 0(R2)
        ADDI    R1, R1, #1
        SW      0(R2), R1
        ADDI    R2, R2, #4
        SUB     R4, R3, R2
        BNZ     R4, LOOP
```

Memory[R2] += 1

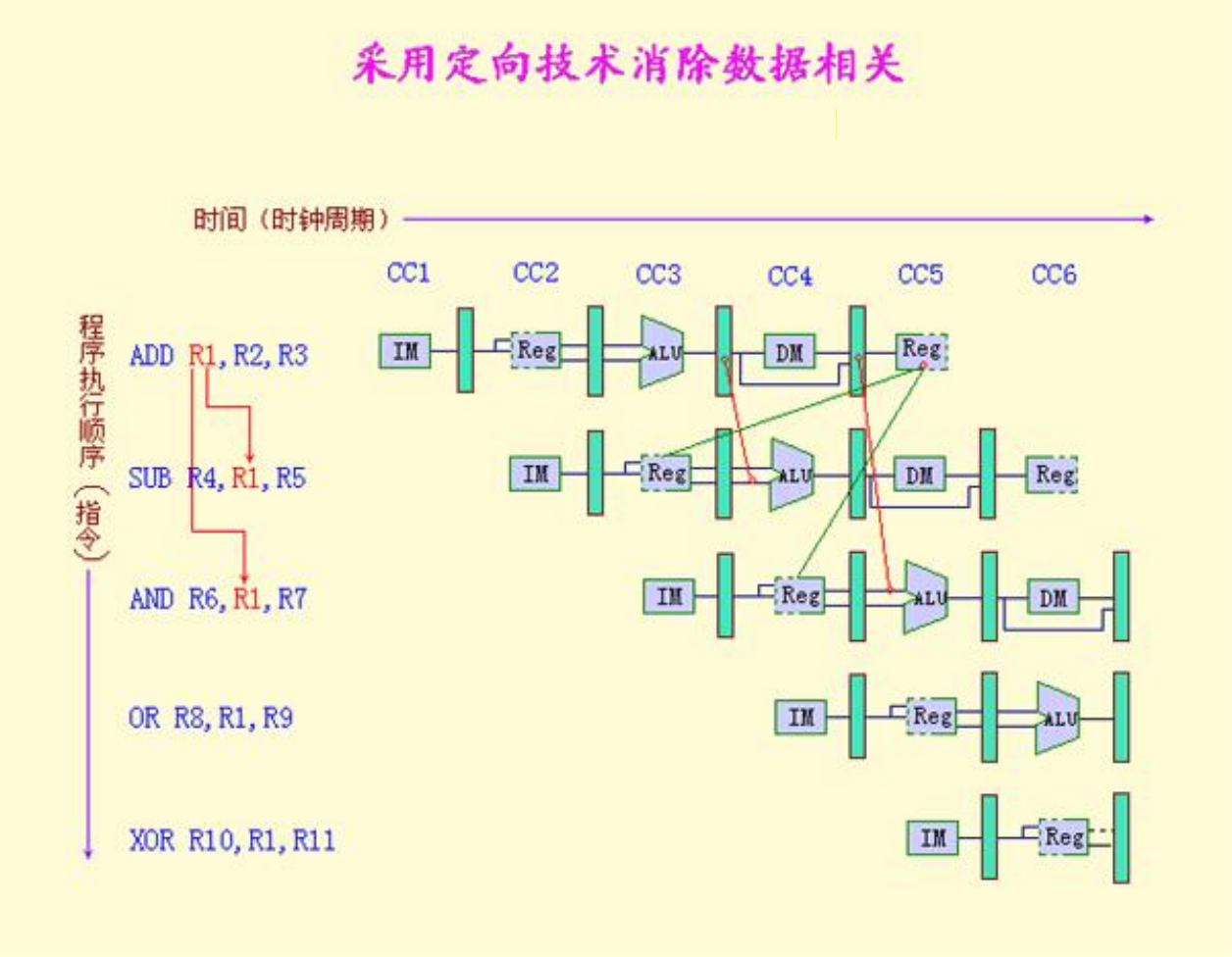
其中，R3的初始值是R2＋396。假设：在整个代码序列的运行过程中，所有的存储器访问都是命中的，并且在一个时钟周期中对同一个寄存器的读操作和写操作可以通过寄存器“定向”。问：

（1）在没有任何其它定向硬件的支持下，请画出该指令序列执行的流水线时空图。假设采用排空流水线的策略处理分支指令，且所有的存储器访问都可以命中Cache，那么执行上述循环需要多少个时钟周期？

（2）假设该流水线有正常的定向路径，请画出该指令序列执行的流水线时空图。假设采用预测分支失败的策略处理分支指令，且所有的存储器访问都可以命中Cache，那么执行上述循环需要多少个时钟周期？

（3）假设该流水线有正常的定向路径，请对该循环中的指令进行调度。注意可以重新组织指令的顺序，也可以修改指令的操作数，但是不能增加指令的条数。请画出该指令序列执行的流水线时空图，并计算执行上述循环需要的时钟周期数？

采用定向技术消除数据相关



习题3.11（1）

(1) 未改进方案：从每轮循环第1条指令开始到下轮循环第1条指令开始（此时最后一条指令bnez r4,Loop才执行完ID 周期）为15拍，见下图。末轮循环的最后一条指令（bnez r4,Loop）在ID周期后还执行3 拍才结束.

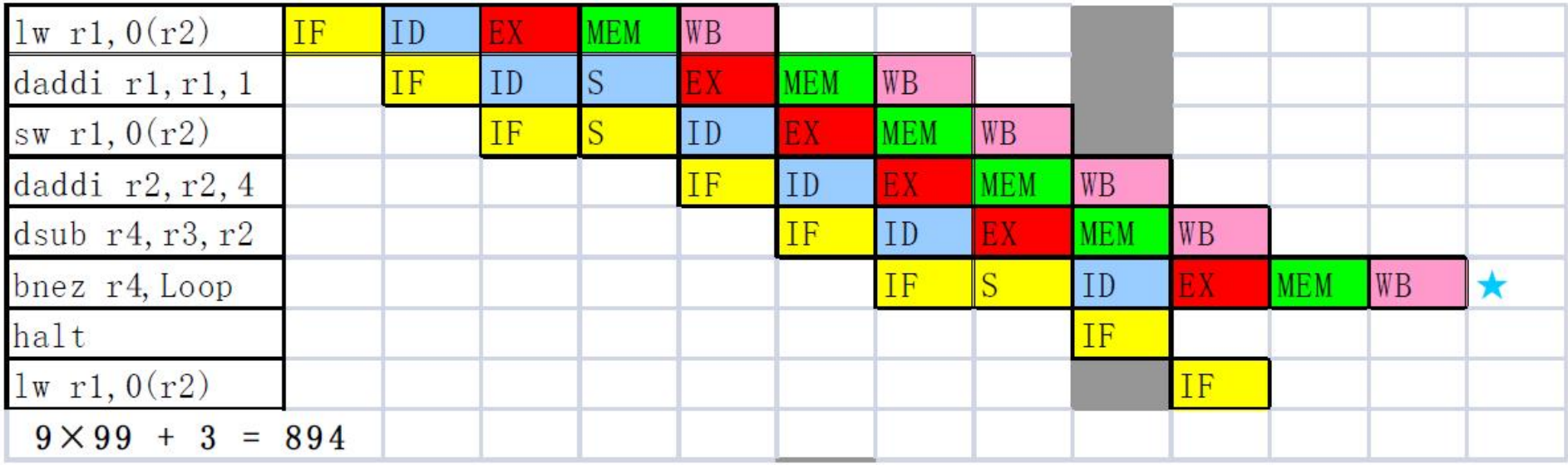
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
lw r1,0(r2)	IF	ID	EX	MEM	WB														
daddi r1,r1,1		IF	S	S	ID	EX	MEM	WB											
sw r1,0(r2)					IF	S	S	ID	EX	MEM	WB								
daddi r2,r2,4								IF	ID	EX	MEM	WB							
dsub r4,r3,r2									IF	S	S	ID	EX	MEM	WB				
bnez r4,Loop												IF	S	S	ID	EX	MEM	WB	★
halt															IF				
lw r1,0(r2)																IF	ID	EX	
15×99 + 3 = 1488																			

需要进行396/4=99次循环，由于每次分支都清空流水线。从上图可以看出每次循环需要15个时钟周期，因此总共需要的时钟周期数= 15×99 + 3 = 1488

习题3.11（2）

(2)采用定向技术，按“预测分支失败”的策略处理分支指令的。

从每轮循环第1 条指令开始到下轮循环第1 条指令开始为9拍，见下图。末轮循环的最后一条指令（bnez r4,Loop）在ID周期后还执行3拍才结束。

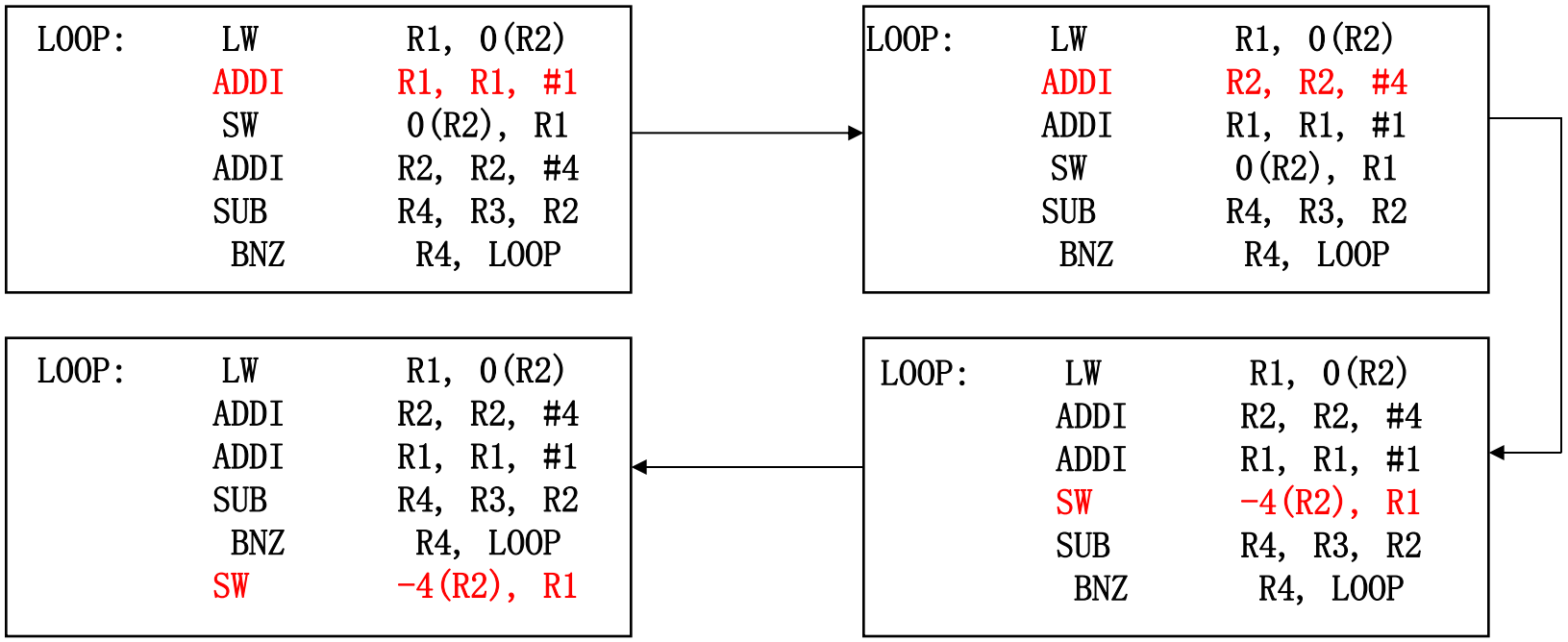


需要进行396/4=99次循环，由于每次分支都清空流水线。从上图可以看出每次循环需要9个时钟周期，因此总共需要的时钟周期数为9×99 + 3 = 894

习题3.11（3）

指令执行重新排序如下：

```
lw      r1,0(r2)        ;加法寄存器R1←取数(R2)
addi    r2,r2,#4        ;指针R2←指针R2+4
addi    r1,r1,#1        ;R1←R1+1
Sub r4,r3,r2            ;R4←R3-R2
bnez    r4,Loop                 ;若R4≠0，循环
sw      -4(r2),r1               ;分支延迟槽，存数(R2-4)←R1
```



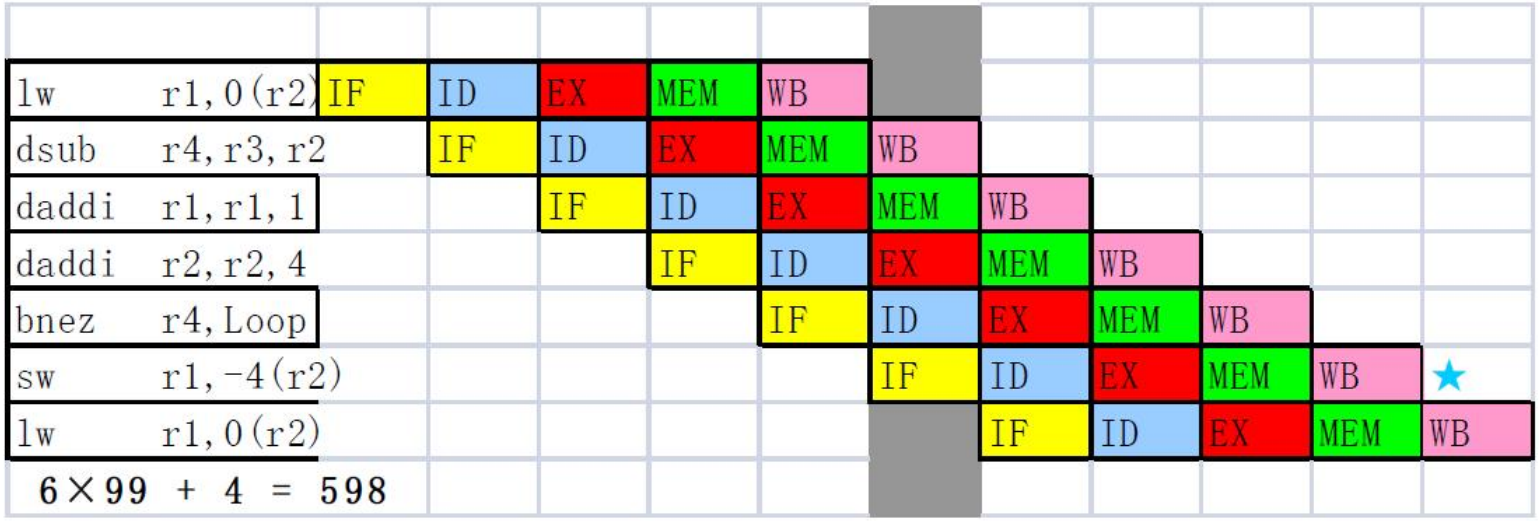
习题3.11（3）

采用定向技术、单周期延迟分支、指令调度技术，重新安排指令顺序如下：

```
loop: lw   r1, 0(r2)
addi   r2,r2,#4
addi  r1,r1,#1
sub  r4,r3,r2
bnz  r4,loop
sw   r1,-4(r2)
```

从每轮循环第1 条指令开始到下轮循环第1 条指令开始为6 拍，见下图。末轮循环的最后一条指令（sw -4(r2),r1）在ID周期后还执行4 拍才结束.

所以，总拍数= 6×99 + 4 = 598



流水线基础

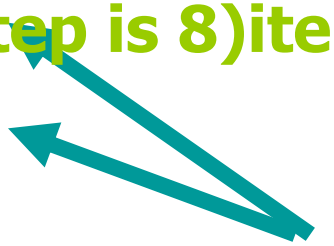
Appendix A.2

Use the following code fragment:

```
Loop: LD      F0,0(R2)
      LD      F4,0(R3)
      MUL.D   F0,F0,F4
      ADD.D   F2,F0,F2
      DADDUI  R2,R2,#8
      DADDUI  R3,R3,#8
      DSUBUR5,R4,R2
      BNEZ    R5,Loop
```

792/8 = 99

(step is 8)iteration = 99 tim



- Assume that the initial value of R4 is R2+792

流水线基础

- For this exercise assume the standard five stage integer pipeline and the MIPS FP pipeline as described in section A.5. If structural hazards are due to write-back contention, assume the earliest instruction gets priority and other instructions are stalled.

  Qa. Show the timing of this instruction sequence for the MIPS FP pipeline without any forwarding or bypassing hardware but assuming a register read and a write in the same clock cycle "forward" through the register file. Assume that the branch is handled by flushing the pipeline. If all memory references hit in the cache, how many cycles does this loop take to execute?

流水线基础

- Qb. Show the timing of this instruction sequence for the MIPS FP pipeline with normal forwarding or bypassing hardware. Assume that the branch is handled by predicting it as not taken. If all memory references hit in the cache, how many cycles does this loop take to execute?
- See 3 Hazards & Forwarding
  - Structural Hazard
  - Data Hazard
  - Control Hazard

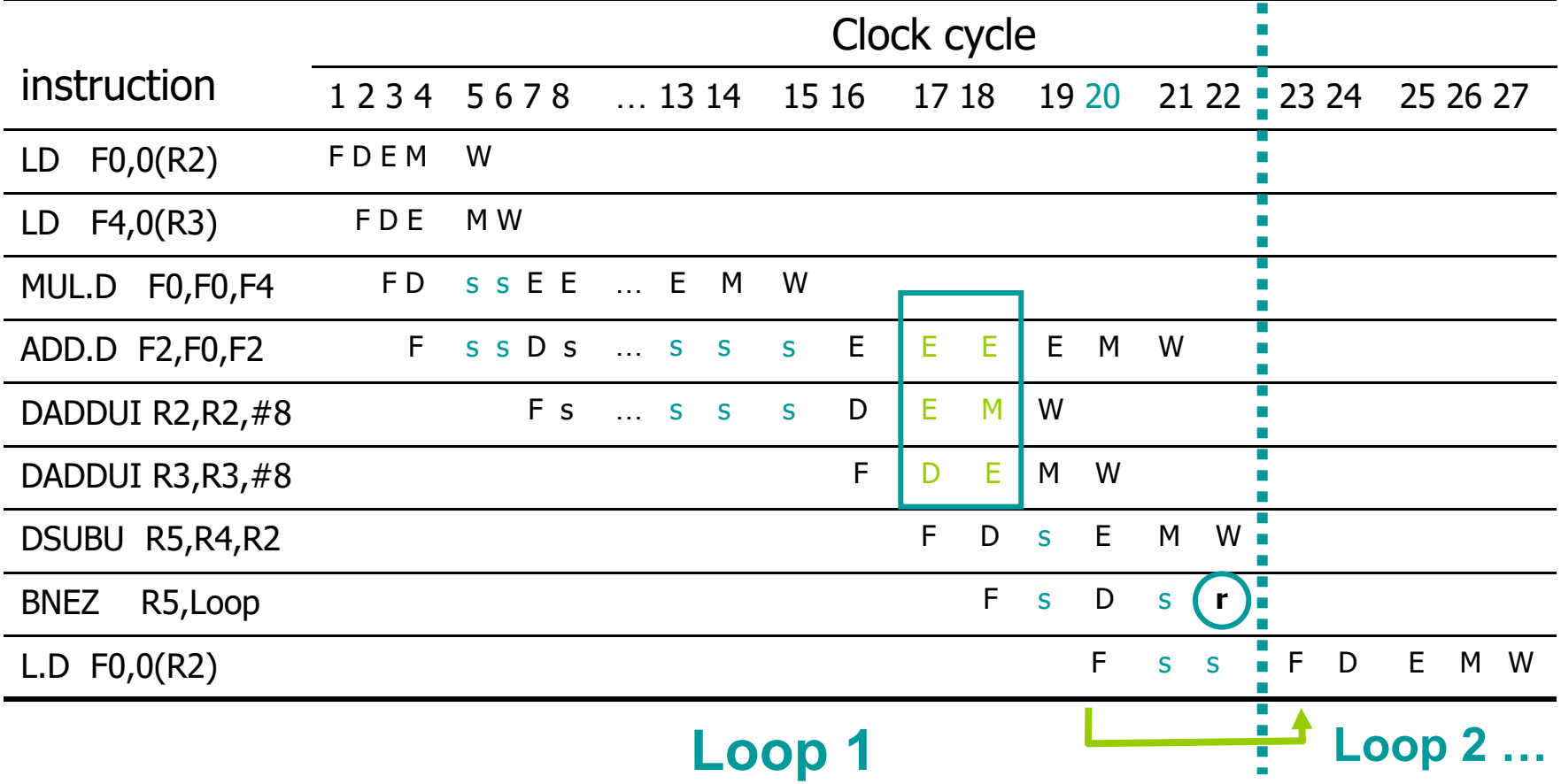
流水线基础

✦ Answer a. (without forwarding)

instruction	Clock cycle																									
	1	2	3	4	5	6	7	8	…	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
LD F0,0(R2)	F	D	E	M	W																			
LD F4,0(R3)		F	D	E	M	W																		
MUL.D F0,F0,F4			F	D	s	s	E	E	…	E	M	W												
ADD.D F2,F0,F2				F	s	s	D	s	…	s	s	s	E	E	E	E	M	W						
DADDUI R2,R2,#8							F	s	…	s	s	s	D	E	M	W								
DADDUI R3,R3,#8													F	D	E	M	W							
DSUBU R5,R4,R2														F	D	s	E	M	W					
BNEZ R5,Loop															F	s	D	s	r					
L.D F0,0(R2)																	F	s	s	F	D	E	M	W

流水线基础

✦ Answer a. (without forwarding)



流水线基础

- Answer a.

  total loop execution time

  = 22 × 99 = 2178 clock cycles

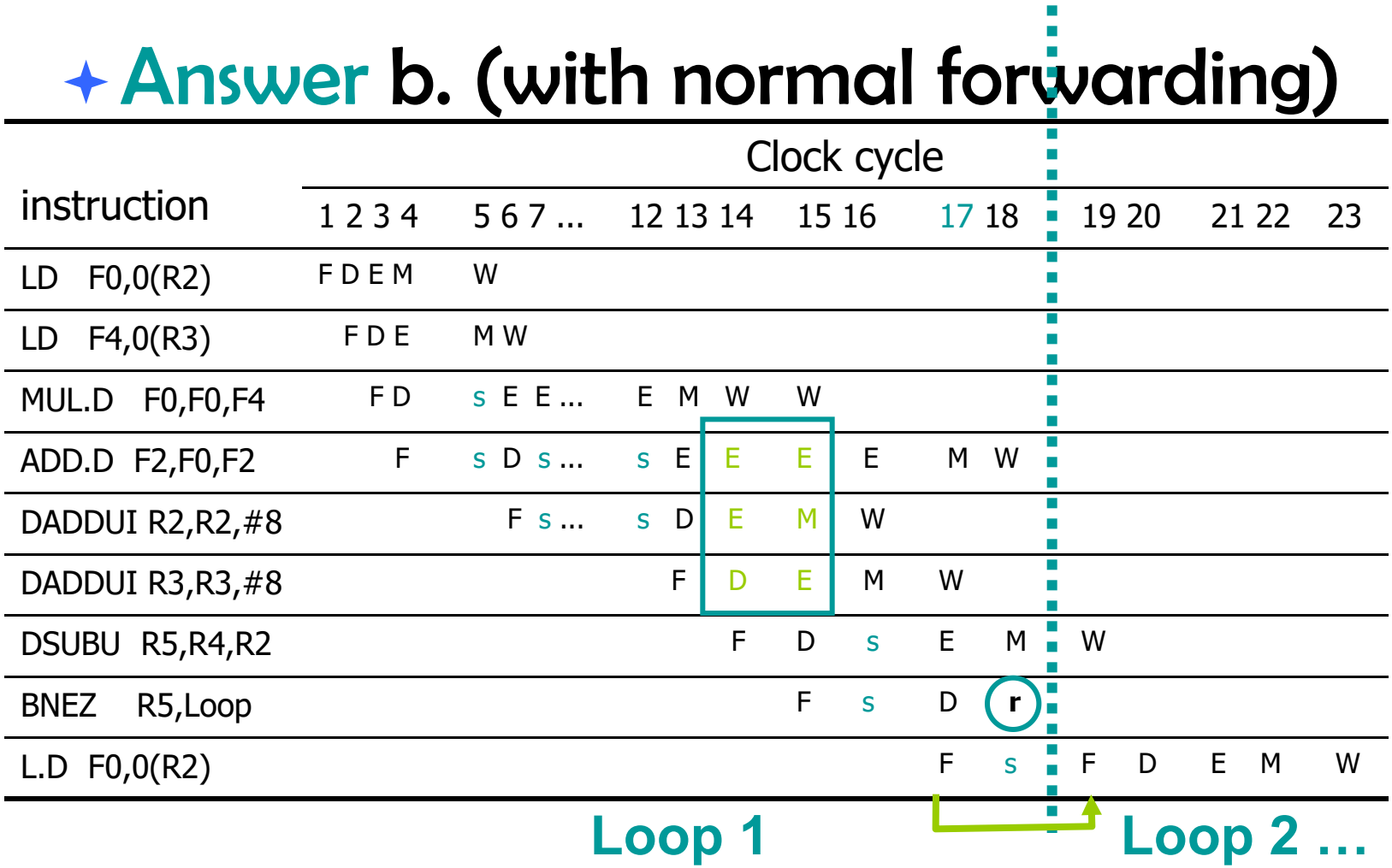
流水线基础

✦ Answer b. (with normal forwarding)

instruction	Clock cycle																							
	1	2	3	4	5	6	7	...	12	13	14	15	16	17	18	19	20	21	22	23				
LD F0,0(R2)	F	D	E	M	W																			
LD F4,0(R3)		F	D	E	M	W																		
MUL.D F0,F0,F4			F	D	s	E	E	...	E	M	W													
ADD.D F2,F0,F2				F	s	D	s	...	s	E	E	E	E	M	W									
DADDUI R2,R2,#8						F	s	...	s	D	E	M	W											
DADDUI R3,R3,#8										F	D	E	M	W										
DSUBU R5,R4,R2											F	D	s	E	M	W								
BNEZ R5,Loop												F	s	D	r									
L.D F0,0(R2)														F	s	F	D	E	M	W				

流水线基础

✦ Answer b. (with normal forwarding)



流水线基础

- Answer b.

  total loop execution time

  = 18 × 98 + 19 = 1783 clock cycles

流水线基础

- Appendix A.3

  Suppose the branch frequencies (as percentage of all instructions) are as follows:

  - Conditional branches 15%
  - Jumps & Calls 1%
  - Conditional branches 60% are taken

  We are examining a four-deep pipeline where the branch is resolved at the end of the second cycle for unconditional branches and at the end of the third cycle for conditional branches. Assuming that only the first pipe stage can always be done independent of whether the branch goes and ignoring other pipeline stalls, how much faster would the machine be without any branch hazards?

流水线基础

Answer

- Pipeline CPI = Ideal pipeline CPI + (Structural Hazard Stalls + Data Hazard Stalls + Control Hazard Stalls)

$$\text{Pipeline speedup} = \frac{\text{Pipeline Depth}}{1 + \text{Pipeline stalls}}$$

- No Control Hazard:

  Pipeline $\text{speedup}_{\text{ideal}} = 4/(1+0) = 4$

流水线基础

- Having Control Hazard:
  Assume 4 stage: IF,ID,EX and WB
  - **Handle Jump & Call:**

Instruction	Clock cycle					
	1	2	3	4	5	6
Jump or Call	IF	ID	EX	WB		
i+1		~~IF~~	IF	ID	EX	…
i+2			stall ⟷	IF	ID	…
i+3				stall	IF	…

流水线基础

- **Handle taken conditional branch:**

Instruction	Clock cycle					
	1	2	3	4	5	6
Taken Branch	IF	ID	EX	WB		
i+1		~~IF~~	stall	IF	ID	…
i+2			stall ⟷	stall ⟷	IF	…
i+3				stall	stall	…

流水线基础

- **Handle not-taken conditional branch:**

Instruction	Clock cycle					
	1	2	3	4	5	6
Not-taken Branch	IF	ID	EX	WB		
i+1		IF	stall	ID	EX	…
i+2			stall	IF	ID	…
i+3				stall	IF	…

流水线基础

- Summary of above 3 control flow instructions:

Control flow type	Frequency (per instruction)	Stall (cycles)
Jump & Call	1%	1
Conditional (taken)	15%×60%=9%	2
Conditional (not taken)	15%×40%=6%	1

$$\text{Pipeline Stall}_{real} = (1\times 1\%) + (2\times 9\%) + (1\times 6\%) = 0.25$$

$$\text{Pipeline Speedup}_{real} = 4/(1+0.25) = 3.2$$

流水线基础

- Pipeline Speedup$_{\text{without control hazard}}$ = 4/3.2 = 1.25

  —— 25% speedup

第四章 存储系统

- **存储层次结构基础**

性能评估指标

1. **命中率（Hit Rate）**：H = 命中次数 / 总访问次数，失效率 = 1 - H。
2. **平均访存时间（AMAT）**：AMAT = 命中时间 + 失效率 × 失效开销。
3. **多级存储**：L1 Cache 命中时间短（1-3 周期），L2/L3 容量大，全局失效率 = L1 失效率 ×L2 失效率。

Cache

地址映射与变换

1. **直接映射**：主存块 i 映射到 Cache 块 i mod M（M 为 Cache 块数），硬件简单但冲突率高。
2. **组相联映射**：主存块映射到固定组，组内全相联，如 N 路组相联，组号 = i mod 组数，降低冲突率。
3. **全相联映射**：主存块可放任意 Cache 块，失效率最低但硬件复杂。

替换算法

1. **随机（Random）**：简单但失效率高。
2. **先进先出（FIFO）**：按进入顺序替换，未利用局部性。
3. **最近最少使用（LRU）**：替换最长未访问块，失效率低但硬件实现复杂，可用计数器或比较对法近似。

写策略

1. **写直达（Write-Through）：写操作同时更新 Cache 和主存，一致性好，速度慢。**
2. **写回（Write-Back）：写操作仅更新 Cache，块替换时写回主存，速度快，需维护脏位。**
3. **写分配与非写分配：写失效时是否将块调入 Cache，写回法常搭配写分配。**

- **性能优化技术**
  - 3C

（一）降低失效率

1. **增大块大小：**减少强制性失效，但可能增加冲突失效，需平衡块大小与失效开销。
2. **提高相联度：**减少冲突失效，如从直接映射到 8 路组相联，失效率显著下降。
3. **Victim Cache：**小容量全相联 Cache 存储被替换的块，捕获冲突失效，如 4 项 Victim Cache 可减少 20%-90% 冲突失效。
4. **编译器优化**
   - **循环交换：**调整循环顺序利用空间局部性，如矩阵转置优化。
   - **分块技术：**将大矩阵分块处理，如矩阵乘法分块到 Cache，减少跨块访问。
   - **数组合并：**合并频繁访问的数组，减少 Cache 行冲突。

（二）减少失效开销

1. **多级 Cache**
   - **L1+L2 结构：**L1 小而快（如 8KB），L2 大而慢（如 256KB），全局失效率 = L1 失效率 ×L2 失效率。
   - **性能计算：**AMAT = 命中时间L1 + 失效率L1×(命中时间L2 + 失效率L2×失效开销)。
2. **关键数据优先：**先传输 CPU 急需的字，提前重启流水线，减少等待时间。
3. **写缓冲合并：**合并连续写操作到同一主存块，减少访存次数。

（三）缩短命中时间

1. **小而简单的 L1 Cache：**如 Alpha 21164 采用 8KB 直接映射 L1，命中时间 1 周期。
3. **路预测（Way Prediction）：**预测访问的 Cache 路，减少比较次数，如两路组相联预测准确率超 90%。
4. **虚拟索引物理标签（VIPT）：**用虚拟地址索引 Cache，物理地址标签比较，避免地址转换延迟。

（四）高级优化技术

1. **无阻塞 Cache：**允许 Cache 失效时继续处理其他请求，支持 “命中下失效” 和 “多重失效”。
4. **硬件预取：**自动预取后续数据，如 Alpha 21064 预取 2 块指令，捕获 15%-72% 失效。
5. **编译器控制预取：**插入预取指令，如循环中提前预取后续数据，减少失效等待。

虚拟存储

- 提供大于物理内存的逻辑地址空间，支持多进程隔离与保护，实现内存抽象。
- 段、页、段页

1. **页表结构**
   1. **单级页表**：虚拟页号直接索引页表，适用于小地址空间。
   2. **多级页表**：如二级页表，虚拟地址分多级索引，减少页表占用内存。
   3. **反置页表**：按物理页存储，通过哈希查找，节省空间但查询复杂。
2. **TLB（转换旁路缓冲器）**
   1. **作用**：缓存近期使用的页表项，加速地址转换，命中时间 1 周期。
   2. **结构**：如 Alpha 21064 的 32 项全相联 TLB，存储 VPN→PPN 映射。
3. 加快地址变换方法

典型计算与例题分析

（一）Cache 性能计算

公式：

平均访存时间 = 命中时间 + 失效率 × 失效开销

例：L1 命中时间 1 周期，失效率 5%，失效开销 50 周期，L2 命中时间 10 周期，失效率 20%，主存访问时间 100 周期。

计算：

AMAT = 1 + 5%×(10 + 20%×100) = 1 + 0.05×30 = 2.5 周期

（二）虚拟存储页表计算

问题：4GB 虚拟地址空间，页大小 4KB，二级页表，一级页表占 1 页，计算二级页表项数。

解：

- 虚拟页号位数：log2(4GB/4KB) = 20位
- 一级页表项数：4KB/4B = 1K（1 页存一级页表）
- 二级页表项数：1K × 1K = 1M（每个一级页表项指向一个二级页表）

（三）多体交叉存储带宽计算

例：4 体交叉存储，存储周期 80ns，总线宽度 64 位，计算最大带宽。

解：

- 每体访问间隔：80ns/4 = 20ns
- 带宽：64bit/20ns = 3.2GB/s

And ….

5.2 简述“Cache—主存”层次与“主存—辅存”层次的区别。

答：

存储 / 层次 / 比较项目	“Cache—主存”层次	“主存—辅存”层次
目的	为了弥补主存速度的不足	为了弥补主存容量的不足
存储管理的实现	全部由专用硬件实现	主要由软件实现
访问速度的比值（第一级比第二级）	几比一	几万比一
典型的块（页）大小	几十个字节	几百到几千个字节
CPU 对第二级的访问方式	可直接访问	均通过第一级
不命中时 CPU 是否切换	不切换	切换到其它进程

5.3 地址映象方法有哪几种？它们各有什么优缺点？

答：(1) 全相联映象。实现查找的机制复杂，代价高，速度慢。Cache 空间的利用率较高，块冲突概率较低，因而 Cache 的失效率也低。（2）直接映象。实现查找的机制简单，速度快。Cache 空间的利用率较低，块冲突概率较高，因而 Cache 的失效率也高。（3）组相联映象。组相联是直接映象和全相联的一种折衷。

5.6 通过编译器对程序优化来改进 Cache 性能的方法有哪几种？简述其基本思想。

答：（1）数组合并。通过提高空间局部性来减少失效次数。有些程序同时用相同的索引来访问若干个数组的同一维，这些访问可能会相互干扰，导致冲突失效，可以将这些相互独立的数组合并成一个复合数组，使得一个 Cache 块中能包含全部所需元素。（2）内外循环交换。循环嵌套时，程序没有按数据在存储器中的顺序访问。只要简单地交换内外循环，就能使程序按数据在存储器中的存储顺序进行访问。（3）循环融合。有些程序含有几部分独立的程序段，它们用相同的循环访问同样的数组，对相同的数据作不同的运算。通过将它们融合成一个单一循环，能使读入 Cache 的数据被替换出去之前得到反复的使用。（4）分块。通过改进时间局部性来减少失效。分块不是对数组的整行或整列进行访问，而是对子矩阵或块进行操作。

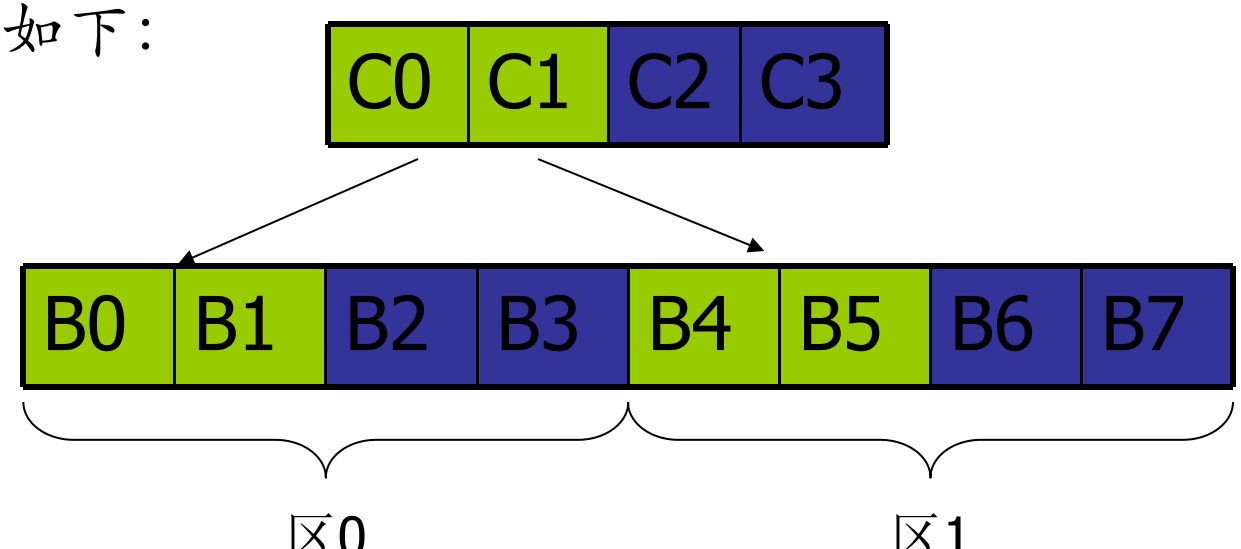
存储系统

假设在一个采用组相联映象方式的Cache中，主存由B0-B7共8块组成，Cache有2组，每组2块，每块的大小为16个字节，采用LRU块替换算法。

- 在一个程序执行过程中依次访问这个Cache的块地址流如下：

  B6，B2，B4，B1，B4，B6，B3，B0，B4，B5，B7，B3

- 假设主存与Cache之间的各个块的映象的对应关系如下：



存储系统

1. 写出主存地址的格式，并标出各字段的长度
2. 写出Cache的地址格式，并标出各字段的长度
3. 如果Cache的各个块号为C0,C1,C2,C3,列出程序执行过程中Cache的块地址流情况
4. 如果采用LRU替换算法，计算Cache的块命中率
5. 如果采用FIFO替换算法，计算Cache的块命中率

存储系统

1. 写出主存地址的格式，并标出各字段的长度

区号1	组号1	块号1	块内地址（4位）

2. 写出Cache的地址格式，并标出各字段的长度

组号1	块号1	块内地址（4位）

存储系统

3. 如果Cache的各个块号为C0,C1,C2,C3,列出程序执行过程中Cache的块地址流情况

	B6	B2	B4	B1	B4	B6	B3	B0	B4	B5	B7	B3
C0			B4	B4*	B4	B4*	B4*	B4*	B4	B4*	B4*	B4*
C1				B1	B1*	B1*	B1*	B0	B0*	B5	B5*	B5*
C2	B6	B6*	B6*	B6*	B6*	B6	B6*	B6*	B6*	B6*	B7	B7*
C3		B2	B2*	B2*	B2*	B2*	B3	B3*	B3*	B3*	B3*	B3
					★	★			★			★

B6，B2，B4，B1，B4，B6，B3，B0，B4，B5，B7，B3
C2　C3　C0　C1　C0　C2　C3　C1　C0　C1　C2　C3

存储系统

- 4. 如果采用LRU替换算法，计算Cache的块命中率

	B6	B2	B4	B1	B4	B6	B3	B0	B4	B5	B7	B3
C0			B4	B4*	B4	B4*	B4*	B4*	B4	B4*	B4*	B4*
C1				B1	B1*	B1*	B1*	B0	B0*	B5	B5*	B5*
C2	B6	B6*	B6*	B6*	B6*	B6	B6*	B6*	B6*	B6*	B7	B7*
C3		B2	B2*	B2*	B2*	B2*	B3	B3*	B3*	B3*	B3*	B3
					★	★			★			★

命中率为：**4/12=1/3=33.3%**

存储系统

5. 如果采用FIFO替换算法，计算Cache的块命中率

	B6	B2	B4	B1	B4	B6	B3	B0	B4	B5	B7	B3
C0			B4	B4*	B4	B4*	B4*	B0	B0*	B5	B5*	B5*
C1				B1	B1*	B1*	B1*	B1*	B4	B4*	B4*	B4*
C2	B6	B6*	B6*	B6*	B6*	B6	B3	B3*	B3*	B3*	B3*	B3
C3		B2	B2*	B2*	B2*	B2*	B2*	B2*	B2*	B2*	B7	B7*
					★	★						★

命中率为：**3/12=1/4=25%**

页地址流		4	5	3	2	5	1	3	2	2	5	1	3
堆栈内容	$S(1)$	4	5	3	2	5	1	3	2	2	5	1	3
	$S(2)$		4	5	3	2	5	1	3	3	2	5	1
	$S(3)$			4	5	3	2	5	1	1	3	2	5
	$S(4)$				4	4	3	2	5	5	1	3	2
	$S(5)$						4	4	4	4	4	4	4
	$S(6)$												
实页数	$n=1$									H			
	$n=2$									H			
	$n=3$					H				H			
	$n=4$					H		H	H	H	H	H	H
	$n\geqslant 5$					H		H	H	H	H	H	H

模拟结果表明，使用 LRU 替换算法替换，对该程序至少应分配 4 个实页。如果只分配 3 个实页，其页命中率只有 2/12，太低；而分配实页数多于 4 页后，其页命中率不会再有提高。所以，分配给该程序 4 个实页即可，其可能的最高命中率为 $H=7/12$。

两级Cache

假设在3000次访存中，第一级cache不命中110次，第二级cache不命中55次。试问：在这种情况下，该cache系统的局部不命中率和全局不命中率各是多少？

解：

第一级cache不命中率（全局和局部）是110/3000，即3.67%；

第二级cache的局部不命中率是55/110，即50%；

第二级cache的全局不命中率是55/3000，即1.83%。

存储系统性能指标

给定以下的假设，试计算直接映象Cache和两路组相联Cache的平均访问时间以及CPU的性能。由计算结果能得出什么结论？

(1) 理想Cache情况下的CPI为2.0，时钟周期为2ns，平均每条指令访存1.2次；

(2) 两者Cache容量均为64KB，块大小都是32字节；

(3) 组相联Cache中的多路选择器使CPU的时钟周期增加了10%；

(4) 这两种Cache的失效开销都是80ns；

(5) 命中时间为1个时钟周期；

(6) 64KB直接映象Cache的失效率为1.4%，64KB两路组相联Cache的失效率为1.0%。

平均访存时间＝命中时间＋失效率×失效开销

平均访问时间1-路=1*2.0+1.4% *80=3.12ns

平均访问时间2-路=1*2.0*(1+10%)+1.0% *80=3.0ns

两路组相联的平均访存时间比较低

CPU_{time}=（CPU执行周期数+存储停顿周期数）*时钟周期

CPU_{time}=（IC*CPI执行周期数+总访存失效次数*失效开销） *时钟周期

=IC*（CPI执行*时钟周期+每条指令的访存次数*失效率*失效开销*时钟周期）

$CPU_{time\ 1\text{-}way}$=IC(2.0*2+1.2*0.014*80)＝5.344IC

$CPU_{time\ 2\text{-}way}$=IC(2.2*2+1.2*0.01*80)＝5.36IC

相对性能比：5.36/5.344=1.003

直接映象的访存时间是两路组相联的1.04倍，

两路组相联的平均CPU时间是直接映象的1.003倍。

因此这里选择直接映象。

伪相联

伪相联中，假设在直接映象位置没有发现匹配，而在另一个位置才找到数据（伪命中）时，需要1个额外的周期，而且不交换两个Cache中的数据，失效开销为50个时钟周期。

假设 2KB直接映象Cache的总失效率为0.098，2路相联的总失效率为0.076；128KB直接映象Cache的总失效率为0.010，2路相联的总失效率为0.007。

试求：

（1）推导出平均访存的时间公式。

（2）利用（1）中得到的公式，对于2KBCache和128KBCache，重新计算伪相联的平均访存时间。请问哪一种伪相联更快？

命中时间$_{伪相联}$＝命中时间1路＋伪命中率$_{伪相联}$×1

因此 伪命中率$_{伪相联}$

＝命中率2路－命中率1路

＝（1－失效率2路）－（1－失效率1路）

＝失效率1路－失效率2路。

平均访存时间$_{伪相联}$

＝命中时间1路＋（失效率1路－失效率2路）×1＋失效率2路×失效开销2路

将题设中的数据带入计算，得到：

平均访存时间2KB=1+(0.098-0.076)*1+(0.076 *50 ） =4.822

平均访存时间128KB=1+(0.010-0.007)*1+(0.007 *50 ） =1.353

显然是128KB的伪相联Cache要快一些。

TLB

假设采用理想存储器系统时的基本CPI是1.5，主存延迟是40个时钟周期；传输速率为4B/时钟周期，且Cache中50%的块修改过的。每个块中有32B，20%的指令是数据传送指令。并假设没有写缓存，在TLB不命中的情况下需要20时钟周期，TLB不会降低Cache命中率。CPU产生指令地址或Cache不命中时产生的地址有0.2%没有在TLB中找到。

（1）在理想TLB情况下，计算均采用写回法16KB直接映像混合Cache、16KB两路组相联混合Cache和32KB直接映像混合Cache机器的实际CPI。

（2）在实际TLB情况下，用（1）的结果，计算均采用写回法16KB直接映像混合Cache、16KB两路组相联混合Cache和32KB直接映像混合Cache机器的实际CPI。

其中假设16KB直接映像混合Cache、16KB两路组相联混合Cache和32KB直接映像混合Cache的不命中率分别为2.9%、2.2%和2.0%； 25%的访存为写访存。

TLB

方法一：

（1）假设TLB不命中率=0

Cache中50%的块修改过，所以不命中时，替换Cache需要1次从内存取一块，50%次写回一块，共1.5次。

均摊不命中开销=不命中率×1.5 × [40+32B/4B+0×20]

=不命中率×72

只有数据传送指令才会访问内存，其余跳转只操作寄存器

实际CPI1=1.5+1.2×不命中率×72=1.5+不命中率×86.4

带入3种Cache结构的不命中率得：

Cache结构	不命中率	实际CPI
16KB直接混合映像	0.029	4.0056
16KB两路混合映像	0.022	3.4008
32KB直接混合映像	0.020	3.2280

（2）假设TLB不命中率=0.2%

均摊不命中开销= 不命中率×1.5×[40+32B/4B+0.2%×20]

= 不命中率×1.5×48.04=不命中率×72.06

实际CPI2= 1.5+1.2×不命中率×72.06

= 1.5+不命中率×86.472

带入3种Cache结构的不命中率后得

Cache结构	不命中率	实际CPI
16KB直接混合映像	0.029	4.0077
16KB两路混合映像	0.022	3.4024
32KB直接混合映像	0.020	3.2294

TLB

方法二：

CPI=CPI 执行+存储停顿周期数/指令数

存储停顿由下列原因引起：从主存中取指令;

load 和store指令访问数据：由TLB 引起。

因此可以得到如下式子：

$$\frac{\text{存储停顿周期数}}{\text{指令数}}=\frac{\text{取指令停顿}}{\text{指令数}}+\frac{\text{数据访问停顿}+\text{TLB停顿}}{\text{指令数}}$$

$$\frac{\text{停顿周期数}}{\text{指令数}}=\frac{\text{存储访问}}{\text{指令数}}\times\text{失效率}\times\text{失效开销}$$

(1) 对于理想TLB，TLB 失效开销为0。

对于统一Cache，若读/写失效且块是干净的，失效开销为

P 干净数据＝主存延迟＋传输一个块需要使用的时间＝40＋32/4＝48（拍）

若为读/写失效且块是脏的，失效开销为

P 脏数据＝（主存延迟＋传输一个块需要使用的时间）×2＝96（拍）

CPI = 1.5 + (1+20%) ×失效率×(50% ×48 + 50%×96) = 1.5 + 1.2×R×72

$$\frac{TLB停顿}{指令数}=(\frac{TLB访问次数}{指令数})\times TLB失效率\times TLB失效开销$$

设f 数据为数据访问指令频率），Rt和Pt分别是TLB 的失效率和失效开销，Rc是Cache的失效率，在Cache命中时，TLB 不会增加命中时间，因此，对CPI产生影响的TLB停顿/指令数为：

$$\frac{TLB停顿}{指令数}=(1+f_{数据})\times R_c\times R_t\times P_t\times(1+50\%)$$

其中，(1+f 数据)：每条指令的访问内存次数。

由条件得：TLB停顿/指令数=(1+20%)×Rc×0.2%×20×1.5=0.072Rc

Cache写策略

假设一台计算机具有以下特性：

95%的访存在Cache中命中；

块大小为两个字，且失效时整个块被调入；

CPU发出访存请求的速率为10^9字/秒；

25%的访存为写访问；

存储器的最大流量为10^9字/秒（包括读和写）；

主存每次只能读或写一个字；

在任何时候，Cache中有30%的块被修改过；

写失效时，Cache采用写分配法。

试对于以下两种情况计算主存频带的平均使用比例。

（1）写直达Cache；

（2）写回法Cache。

写策略

采用按写分配

（1）写直达cache：

读命中，不访问主存；

写命中，更新cache和主存，访问主存一次。

读失效，将主存中的块调入cache中，访问主存两次；

写失效，将要写的块调入cache，访问主存两次，再将修改的数据写入cache和主存，访问主存一次，共三次。上述分析如下表所示：

访问命中	访问类型	频率	访存次数
Y	读	95%*75%=71.3%	0
Y	写	95%*25%=23.8%	1
N	读	5%*75%=3.8%	2
N	写	5%*25%=1.3%	3

一次访存请求最后真正的平均访存次数

=(71.3%*0)+(23.8%*1)+(3.8%*2)+(1.3%*3)＝0.35

已用带宽=0.35×10^9/10^9=35.0%

（2）写回法cache访问命中,有两种情况：

读命中，不访问主存；

写命中，采用写回法，不访问主存。

访问失效,有一个块将被换出，这也有两种情况：

如果被替换的块没有修改过，将主存中的块调入cache块中，访存两次；

如果被替换的块修改过，则首先将修改的块写入主存，需要访存两次；然后将主存中的块调入cache块中，需要访问主存两次，共四次访存。

访问命中	块为脏	频率	访存次数
Y	N	95%*70%=66.5%	0
Y	Y	95%*30%=28.5%	0
N	N	5%*70%=3.5%	2
N	Y	5%*30%=1.5%	4

一次访存请求最后真正的平均访存次数=66.5%*0＋28.5%*0+3.5%*2+1.5%*4=0.13

已用带宽＝0.13×10 ^{9}/10 ^{9}＝13%

补习题 有一个全映射的cache，采用写回策略(被替换时才写回内存)，刚开始时cache为空。有下面有个存储器操作：

Write mem[100]

Write mem[100]

Read mem[200]

Write mem[200]

Write mem[100]

分别求写分配和不按写分配情况下的命中次数和缺失次数。

解：不按写分配情况：地址100的单元不在cache内，而cache对写操作不产生分配。即直接写入内存而不会先调入cache再写入内存。因此第一个写失效，第二个也失效。第三个读自然也失效，但此时把内存中的数据块调入了cache。所以第三次写命中，第四次写缺失。

写分配时(写失效时，先调入cache再改写)，对100和200单元的第一次操作都发生缺失。其所在的块被读进了cache，所以对它们的其它几次操作命中。

第五章 指令级并行及其开发

本章研究：如何利用各种技术来开发更多的指令级并行 （硬件的方法）

- **指令级并行度ILP**
- 开发ILP的方法可以分为两大类
  - 主要基于硬件的动态开发方法（动态调度）

    记分牌动态调度算法

    Tomasulo算法
  - 基于软件的静态开发方法（静态调度）
- 理想流水线的CPI加上各类停顿的时钟周期数：

$$CPI_{流水线} = CPI_{理想} + 停顿_{结构冲突} + 停顿_{数据冲突} + 停顿_{控制冲突}$$

- 动态分支预测技术

分支历史表BHT

分支目标缓冲器BTB

- 超标量处理机
- 超流水线处理机
- 超长指令字（VLIW)处理机

数据相关及其处理技术

流水线时**空图**

超标量流水线**调度策略**及**时空图**

1.按序流出按序完成；2.按序流出无序完成；3.无序流出

ILP,DLP,TLP

并行类型	目标	主要实现技术	典型硬件/示例
ILP	提高单线程指令执行效率	- 流水线（Pipelining） - 多发射（超标量/VLIW） - 乱序执行（OoOE） - 分支预测 - 推测执行	Intel Core（乱序超测）
DLP	单指令并行处理大量数据	- SIMD指令集（SSE/AVX/NEON） - GPU SIMT架构 - 向量处理器 - 数据并行编程模型	x86 AVX-512、N
TLP	多线程/多任务并行提升系统吞吐量	- 多核CPU - 超线程（SMT） - SMP/NUMA系统 - 硬件多线程 - 并行编程模型（OpenMP/MPI）	AMD Ryzen（多核块）、OpenMP多

指令级并行

- Consider the following four MIPS code fragments each containing two instructions:

i.	DADDI	R1,R1,#4
	LD	R2,7(R1)
ii.	DADD	R3,R1,R2
	SD	R2,7(R1)
iii.	SD	R2,7(R1)
	SD	F2,200(R7)
iv.	BEZ	R1,place
	SD	R1,7(R1)

指令级并行

- a. For each fragment (i) to (iv) identify each type of dependence that exists or that may exist (a fragment may have no dependence) and describe what data flow, name reuse, or control structure causes or would cause the dependence. For a dependence that may exist, describe the source of the ambiguity and identify the time at which that uncertainty is resolved.
- b. For each code fragment, discuss whether dynamic scheduling is, may be, or is not sufficient to allow out-of-order execution of the fragment.

指令级并行

- Answer

Code fragment	Data Dependence?	Dynamic scheduling sufficient for out-of-order execution?
DADDI R1,R1,#4 LD R2,7(R1)	True dependence of R1	No. Changing instruction order will break program semantics
DADD R3,R1,R2 SD R2,7(R1)	None	Yes
SD R2,7(R1) SD F2,200(R7)	Output dependence may exist	Maybe. If the hardware computes the effective addresses early enough, then the store order may be exchanged.
BEZ R1,place SD R1,7(R1)	None	No. Changing instruction order is speculative until the branch resolved

指令级并行

List all the dependences (output, anti, and true) in the following code fragment. Indicate whether the true dependences are loop carried or not. Show why the loop is not parallel.

- for (i=2;i<100;i=i+1){

```
        a[i]=b[i]+a[i];  /*S1*/
        c[i-1]=a[i]+d[i];     /*S2*/
        a[i-1]=2*b[i];        /*S3*/
        b[i+1]=2*b[i];        /*S4*/
}
```

指令级并行

- Answer:
- There are six dependences in the loop:

1. Anti dependence from S1 to S1 on a
2. True dependence from S1 to S2 on a
3. Loop-carried true dependence from S4 to S1 on b .
4. Loop-carried true dependence from S4 to S3 on b .
5. Loop-carried true dependence from S4 to S4 on b .
6. Loop-carried output dependence from S3 to S1 on a

指令级并行

- The following loop is a dot product (assuming the running sum in F2 is initially 0) and contains a recurrence. Assume the pipeline latencies from Figure 4.1 and a 1-cycle delay branch.

```
foo:  L.D      F0,0(R1);load X[i]
      L.D      F4,0(R2)      ;load Y[i]
      MUL.D    F0,F0,F4      ;multiply X[i]*Y[i]
      ADD.D    F2,F0,F2      ;add sum = sum + X[i]*Y[i]

      DADDUI   R1,R1,#-8     ;decrement X index
      DADDUI   R2,R2,#-8     ;decrement Y index
      BNEZ     R1, foo       ;loop if not done
```

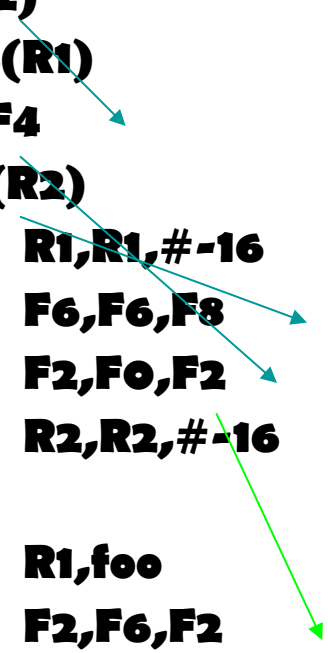
指令级并行

- Qa Assuming a single-issue pipeline. Despite the fact that the loop is not parallel, it can be scheduled with no delays. Unroll the following loop a sufficient number of times to schedule it without any delays. Show the schedule after eliminating any redundant overhead instructions.
  - *Hint*: An additional transformation of the code is needed to schedule without delay.
- Qb Show the schedule of the transformed code from part for the processor in Figure 4.2. For an issue capability that is 100% greater, how much faster is the loop body?

指令级并行

- Answer: simply unrolling twice yields
- foo: **L.D F0,0(R1)**

```
L.D     F4,0(R2)
L.D     F6, #-8(R1)
MUL.D   F0,F0,F4
L.D     F8,#-8(R2)
DADDUI          R1,R1,#-16
MUL.D           F6,F6,F8
ADD.D           F2,F0,F2
DADDUI          R2,R2,#-16
stall
BNEZ            R1,foo
ADD.D           F2,F6,F2
```



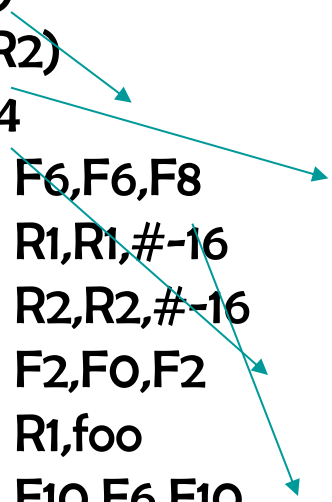
指令级并行

- The dependence chain from one ADD.D to the next ADD.D forces the stall, which is loop-carried. Unrolling further will only aggravate the problem. We must break the dependence chain to schedule without stalls.
- A solution computes the even and odd elements, and store them in F2 and F10 respectively. Combines the two partial sums outside the loop body.
- The loop itself is stall free, but there are three stalls when the loop exist. The loop body takes 11 clock cycles.

指令级并行

```
foo: L.D     F0,0(R1)
     L.D     F6, #-8(R1)
     L.D     F4,0(R2)
     L.D     F8,#-8(R2)
     MUL.D   F0,F0,F4
     MUL.D   F6,F6,F8
     DADDUI  R1,R1,#-16
     DADDUI  R2,R2,#-16
     ADD.D   F2,F0,F2
      BNEZ   R1,foo
     ADD.D   F10,F6,F10

bar: ADD.D   F2,F2,F10
```



b Scheduled code on a superscalar MIPS

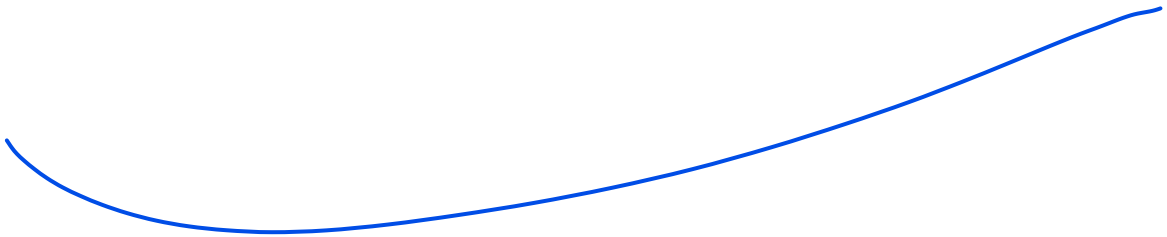
	Integer instruction	FP instruction	Clock cycle
foo:	L.D F0,0(R1)		1
	L.D F6,-8(R1)		2
	L.D F4,0(R2)		3
	L.D F8,-8(R2)		4
	DADDUI R1,R1,#-16	MUL.D F0,F0,F4	5
	DADDUI R2,R2,#-16	MUL.D F6,F6,F8	6
	stall		7
	stall		8
	BNEZ R1,foo	ADD.D F2,F0,F2	9
		ADD.D F10,F6,F10	10
			...
bar:		ADD.D F2,F2,F10	14

The loop body now takes 10 cycles (vs 8 cycles) instead of 11

指令级并行

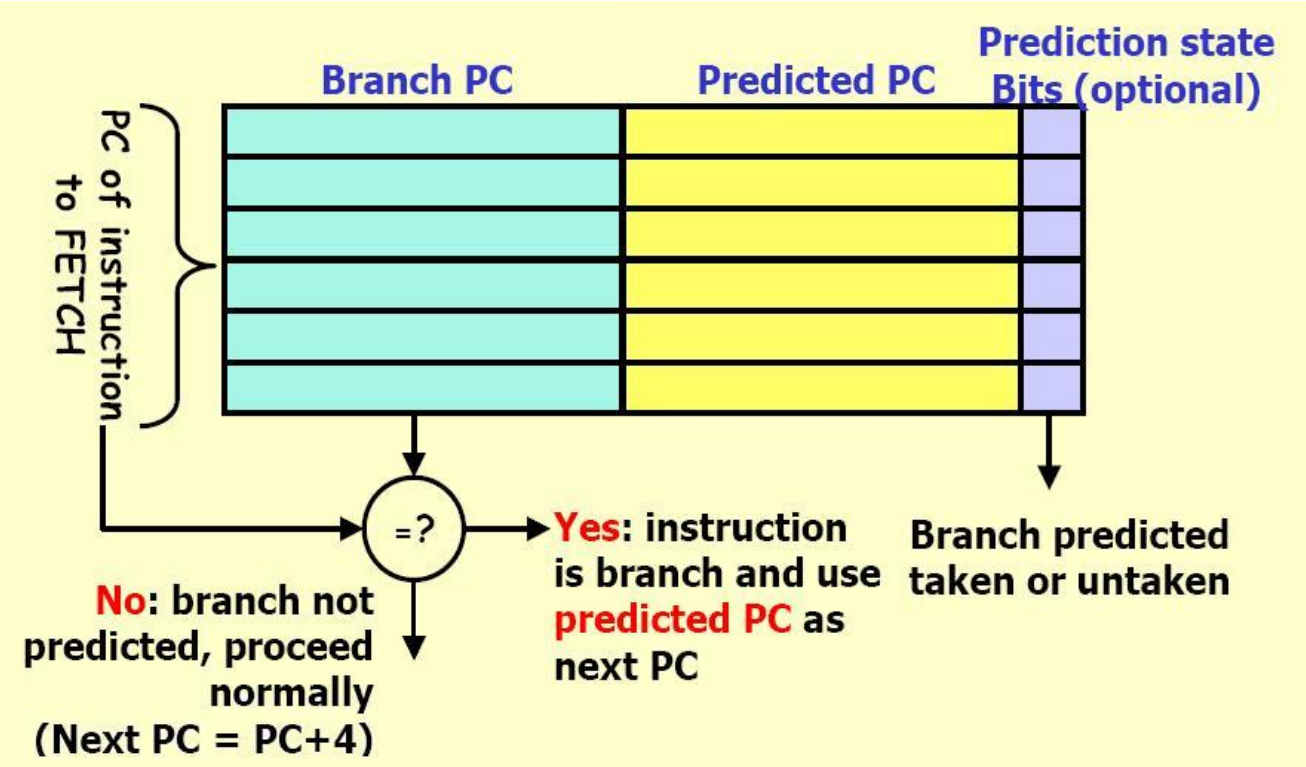
- Suppose we have a deeply pipeline processor. For which we implement a branch-target buffer for the conditional branches only. Assume that the misprediction penalty is always 4 cycles and the buffer miss penalty is always 3 cycles. Assume 90% hit rate and 90% accuracy and 15% branch frequency.

  Q. How much faster is the processor with the branch-target buffer versus a processor that has a fixed 2 cycle branch penalty? Assume a base CPI without branch stall of 1.



指令级并行

- Branch-Target Buffer (BTB):
  Address of branch index to get prediction AND branch address (if taken)



指令级并行

Answer

- CPI_{BTB} - System with a branch-target buffer
  CPI_{NBTB} - System without a branch-target buffer

$$\text{Speedup} = \frac{CPI_{NBTB}}{CPI_{BTB}} = \frac{CPI_{base} + Stall_{NBTB}}{CPI_{base} + Stall_{BTB}}$$

$CPI_{base} = 1$ —— exercise statement

指令级并行

$$Stall = \sum_{s \in Stall} Frequency_s \times Penalty_s$$

$$\therefore \quad Stall_{NBTB} = 15\% \times 2 = 0.3$$

$$Stall_{BTB} = 1.5\% \times 3 + 1.3\% \times 4 = 0.097$$

BTB result	BTB prediction	Frequency (per instruction)	Penalty (cycle)
Miss	--	15%×10%=1.5%	3
Hit	Correct	15%×90%×90%=12.1%	0
Hit	Incorrect	15%×90%×10%=1.3%	4

Assume 90% hit rate and 90% accuracy and 15% branch frequency

指令级并行

$$\text{Speedup} = \frac{CPI_{base} + Stall_{NBTB}}{CPI_{base} + Stall_{BTB}} = \frac{1+0.3}{1+0.097} = 1.2$$

—— 20% faster

6.18 Start

习题5.8（分支预测技术）

假设有一条长流水线，仅仅对条件转移指令使用分支目标缓冲。假设分支预测错误的开销为4个时钟周期，缓冲不命中的开销为3个时钟周期。假设命中率为90%，预测精度为90%，分支频率为15%，没有分支的基本CPI为1。

（1）求程序执行的CPI。

（2）相对固定的2个时钟周期延迟的分支处理，哪种更快？

解：

（1）程序执行的CPI=没有分支的基本CPI+分支带来的额外开销

额外开销=15%*(90%命中*10%预测错误*4+10%没命中*3)

=0.099

所以程序执行的CPI=1+0.099=1.099。

（2）采用固定的2 个时钟周期延迟的分支处理

CPI=1+15%*2=1.3

由（1）（2）知分支目标缓冲方法执行速度快。

习题5.9（分支预测技术）

假定分支目标缓冲的命中率为90%，程序中无条件转移指令为5%，其它指令的CPI为1。假设分支目标缓冲包含分支目标指令,允许无条件转移指令进入分支目标缓冲,则CPI是多少。假定原来的CPI为1.1。

（1）原来不采用分支目标缓冲器BTB情况下

实际CPI = 理想CPI+各种停顿拍数

=1+5%×L=1.1

解出L=2

（2）现在采用分支目标缓冲器BTB情况下

实际CPI=理想CPI+各种停顿拍数

=1+5%×10%×2=1.01

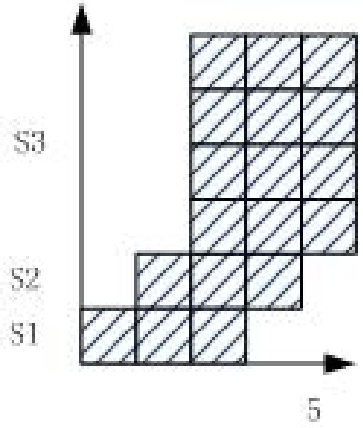
习题5.11（超标量/超长指令字/超流水）

设指令流水线由取指令,分析指令和执行指令3个部件构成,每个部件△t,连续12条指令,分别画出ILP为4的超标量,超长指令字处理机和超流水线的时空图,并分别计算相对标量流水处理机的加速比.

1



部件数+指令数-1



Tk=（k+n-1）△t=(3+12-1）△t=14 △t

2. 超长指令字处理机

采用指令级并行技术，ILP=4, 12个任务组装成3条长指令，每条含4条小指令，n=3。 Tk=（k+n-1）△t=(3+3-1）△t=5 △t，

加速比S= 14 △t/5 △t=2.8

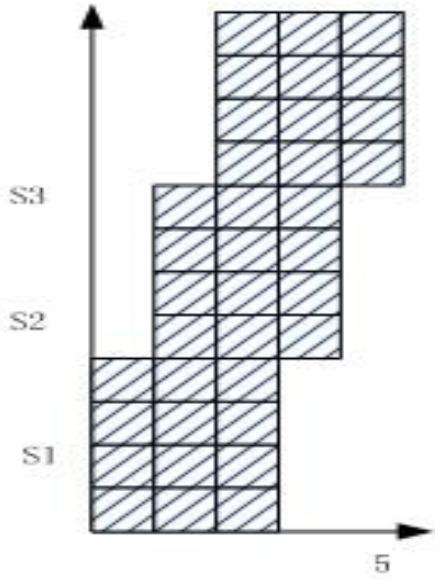
习题5.11

3 ． 超标量处理机

Tk=(k+n-1) △t=(3+3-1) △t

=5 △t

加速比S=14 △t/5 △t=2.8



4. 超流水处理机

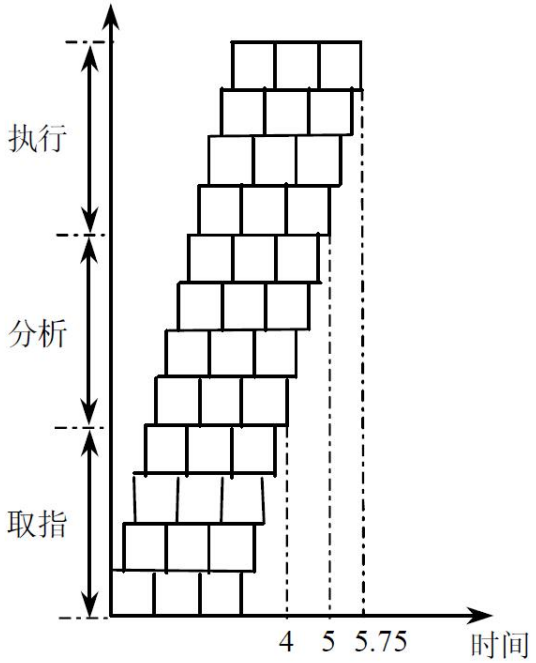
ILP=4,12个任务在4条时钟

依次错开0.25 △t的流水线上流过，

所以可取k=12,n=12, 时钟= △t/4。

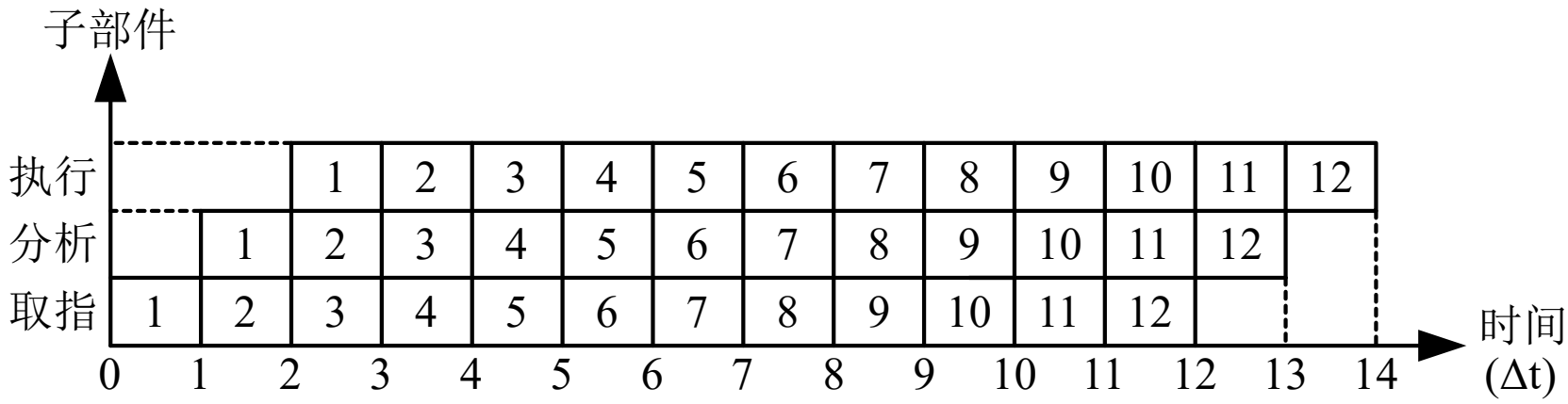
Tk=(k+n-1) △t/4=(12+12-1) △t/4=5.75 △t，

加速比S=14 △t/5 .75 △t=2.435

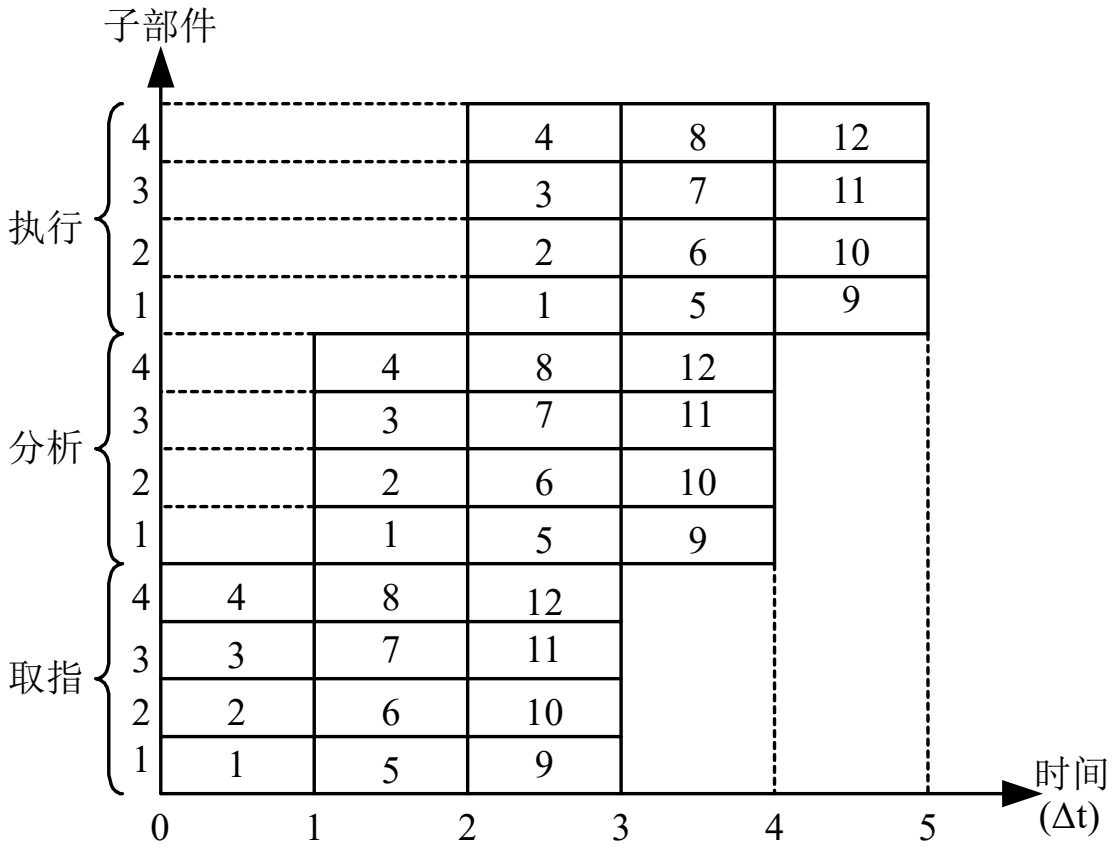


5-17 设指令由取指、分析、执行三个子部件组成。每个子部件经过时间为Δt，连续执行12条指令。请分别画出在常规标量流水处理机及度m均为4的超标量处理机、超长指令字处理机、超流水线处理机上工作的时空图，分别计算它们相对常规标量处理机的加速比SP。

常规标量流水处理机

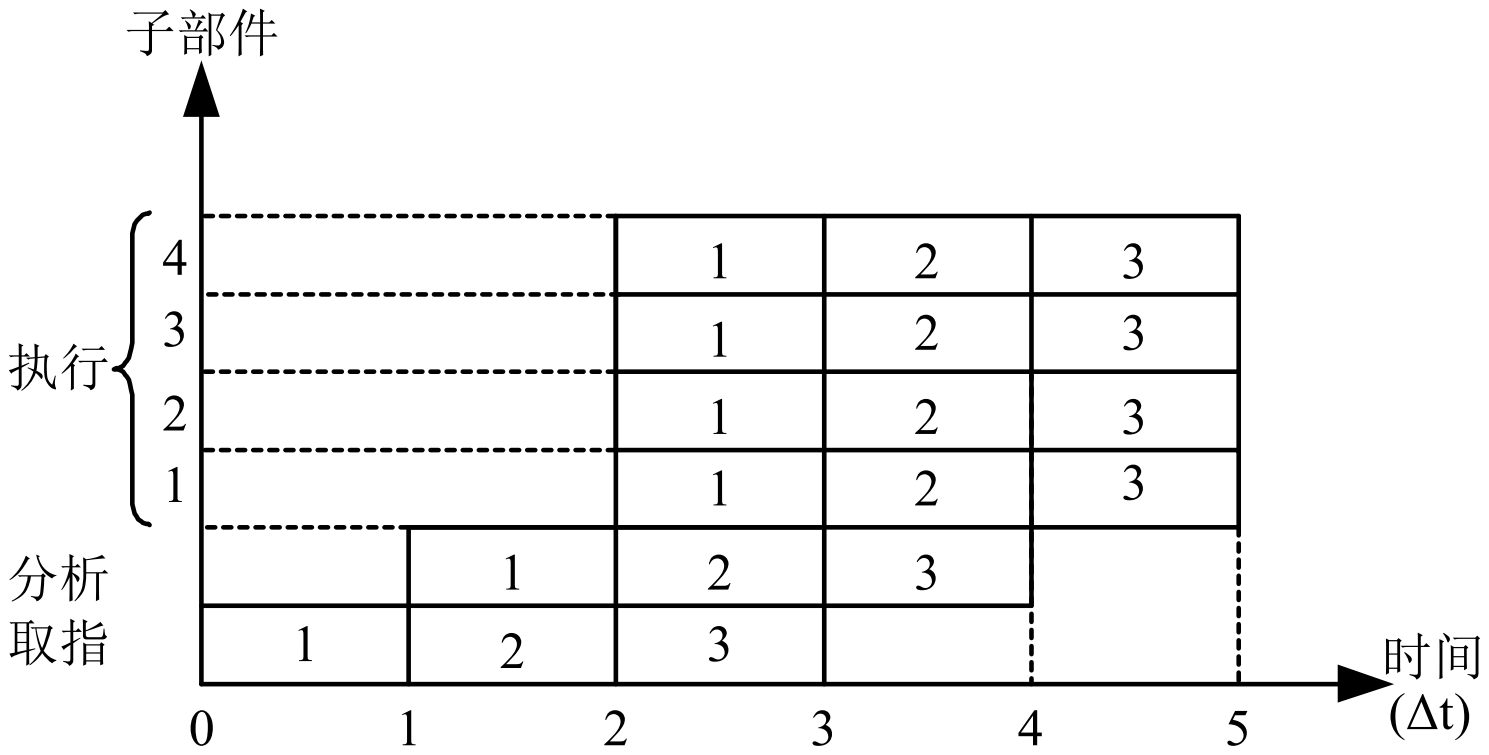

子部件
执行
分析
取指
1 2 3 4 5 6 7 8 9 10 11 12
1 2 3 4 5 6 7 8 9 10 11 12
1 2 3 4 5 6 7 8 9 10 11 12
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
时间
(Δt)

度m=4的超标量处理机



加速比SP=14 Δt /5Δt=2.8

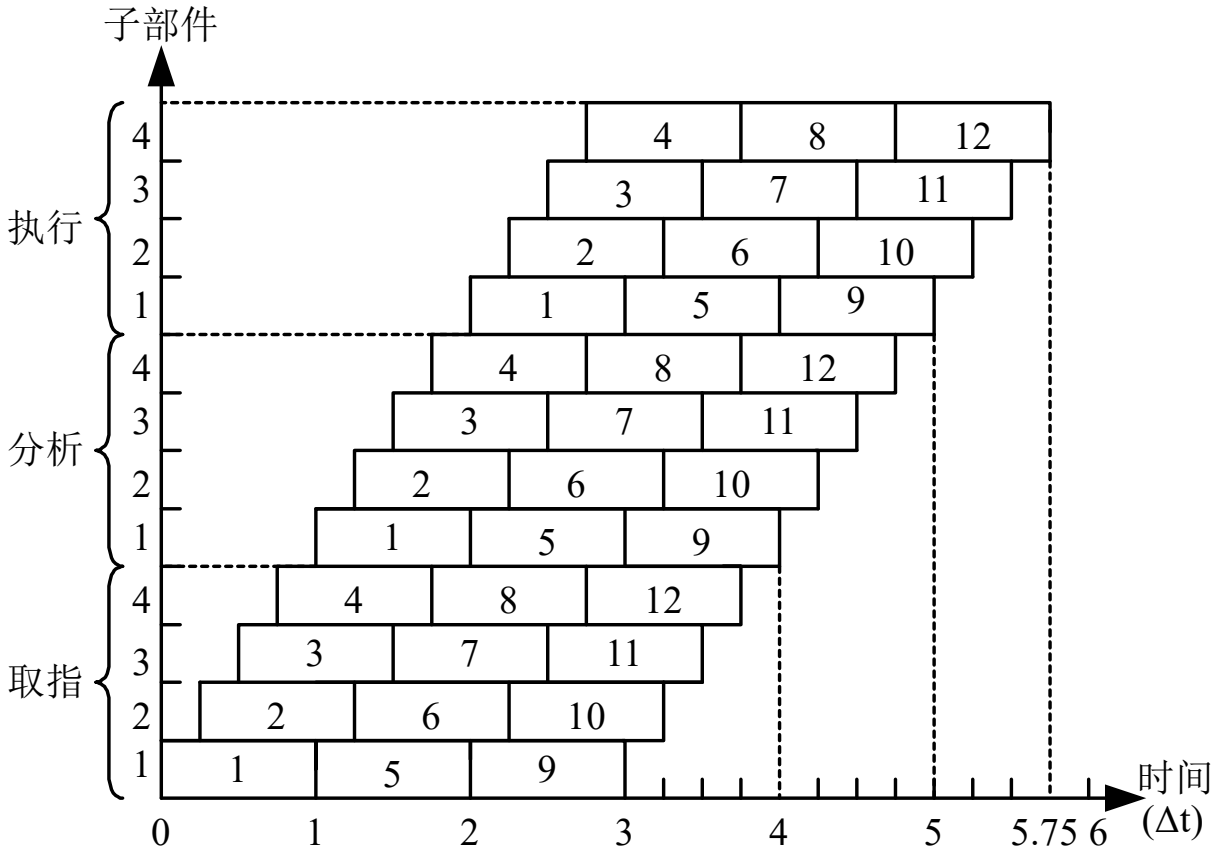
度m=4的超长指令字处理机



加速比SP=14 Δt /5Δt=2.8

第5章

度m=4的超流水线处理机



阶梯状
1/n

加速比SP=14 Δt /5.75Δt =2.43

第6章 数据级并行（DLP）

（一）研究动机

1.传统 ILP 技术瓶颈：指令级并行受限于程序内在并行性、时钟频率提升难度及 Cache 命中率问题。

2.应用驱动：图形处理、机器学习等领域需大规模数值计算，DLP 通过同一操作处理多数据元素提升效率。

3.Flynn 分类：SIMD（单指令多数据流）是 DLP 核心实现方式，代表架构包括向量处理器、SIMD 指令集与 GPU。

（二）DLP 优势

1.能效比：单指令控制多数据操作，减少取指开销，适合移动设备低功耗需求。

2.编程模型：允许程序员保持串行思维，通过向量化编译或硬件自动并行化实现并行。

3.性能提升：向量操作可将循环迭代转化为单指令多数据处理，理论加速比显著。

向量处理方法

横向处理/ 纵向处理/ 纵横处理：向量流水处理机结构

1 存储器-存储器结构

2 寄存器寄存器结构

提高向量处理机性能的方法

多功能部件的并行操作

链接技术（重点）　　只有RAW冒险

分段开采

多通道（Lane）实现：将向量分块至多个并行流水线，如 4 通道处理 32 元素向量，每周期完成 8 操作。

条件执行：向量掩码寄存器控制部分元素操作，如CVM指令激活有效位。

稀疏矩阵处理：通过 Gather/Scatter 指令处理非连续数据，如LVI间接加载稀疏向量。

多处理机系统结构

向量处理机性能的主要参数

- 一条向量指令的处理时间
- 每秒多少个浮点运算结果（MFLOP或一个浮点运算的时间）
- 一组向量指令的处理时间
- 向量流水线的最大性能R∞
- 半性能向量长度n1/2

GPU 架构与编程模型

（一）硬件基础

1.异构系统：CPU+GPU 协同，CPU 处理串行逻辑，GPU 执行数据并行核函数。

2.核心组件

- **流多处理器（SM）**：包含多个 CUDA 核心（SP），如 NVIDIA A100 的 SM 含 64 个 SP。
- **存储层次**：寄存器→共享内存→L1/L2 缓存→全局内存，访问延迟递增。

（二）编程模型

1.CUDA 模型

- **网格（Grid）**：由线程块（Block）组成，如daxpy<<<nblocks, 256>>>指定块数与每块线程数。
- **线程层次**：blockIdx（块索引）、threadIdx（线程索引）映射数据位置。

2.执行模型

- **SIMT（单指令多线程）**：32 线程组成 Warp，同 Warp 执行相同指令，数据不同。
- **分支处理**：发散时用掩码控制活跃线程，如if-else需串行执行两路分支。

维度	向量机	GPU
编程模型	向量指令直接操作数组	多线程 SPMD 模型，CUDA/OpenCL 编程
并行粒度	指令级向量操作	线程级 Warp 并行，动态调度
存储层次	无数据 Cache，依赖多体内存	多级缓存 + 共享内存，优化局部性
典型应用	科学计算（如 Linpack）	机器学习、图形渲染

典型例题与解析

（一）向量处理时间计算

问题：计算 A=B×s（n=200，MVL=64，启动时间 Tstart=31，Tloop=15）。

解答：分段数 = ceil (200/64)=4，总时间 T=4×(15+31)+200×3=784周期，单元素时间 3.9 周期。

（二）GPU 存储器优化

问题：32 线程读取非对齐地址，如何提升带宽？

解答：调整地址为 128B 对齐，利用合并访问将 32×8b 读取合并为 1 次事务，带宽利用率从 1/4 提升至 100%。

3.18 在 CRAY-1 机器上，按照链接方式执行下述 4 条向量指令（括号中给出了相应功能部件的执行时间），如果向量寄存器和功能部件之间的数据传送需要 1 拍，试求此链接流水线的通过时间是多少拍？如果向量长度为 64，则需多少拍才能得到全部结果？

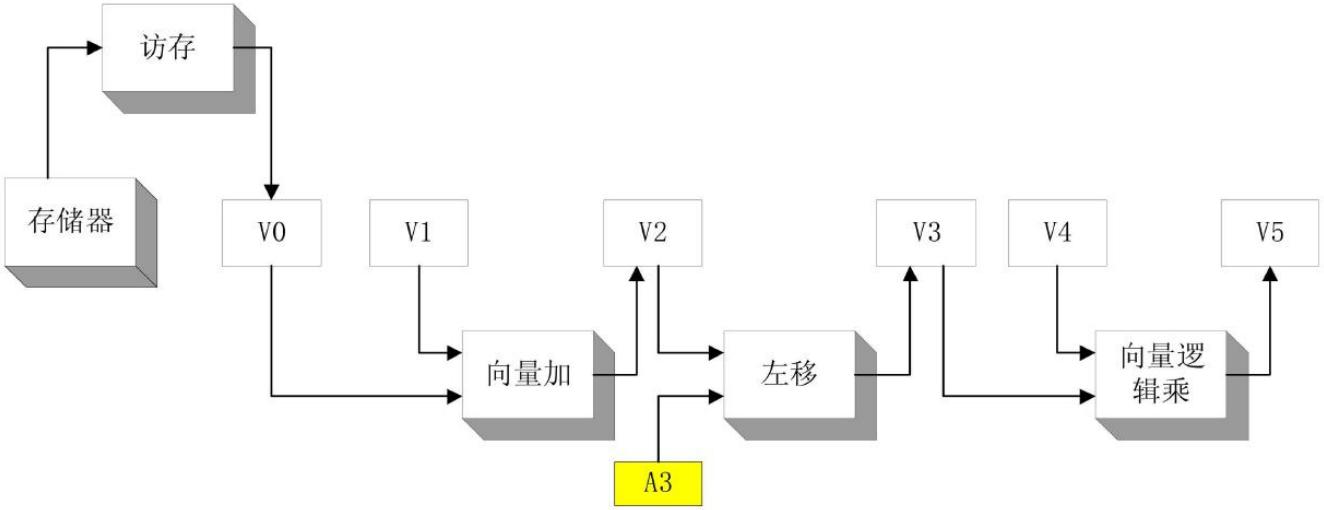
V_0←存储器　　（从存储器中取数：7 拍）

V_2←V_0+V_1　　（向量加：3 拍）

V_3←V_2<A_3　　（按（A_3）左移：4 拍）

V_5←V_3∧V_4　　（向量逻辑乘：2 拍）

解：通过时间就是每条向量指令的第一个操作数执行完毕需要的时间，也就是各功能流水线由空到满的时间，具体过程如下图所示。要得到全部结果，在流水线充满之后，向量中后继操作数继续以流水方式执行，直到整组向量执行完毕。



$T_{通过}$＝（7+1）＋（1+3+1）＋（1+4+1）＋（1+2+1）＝23（拍）

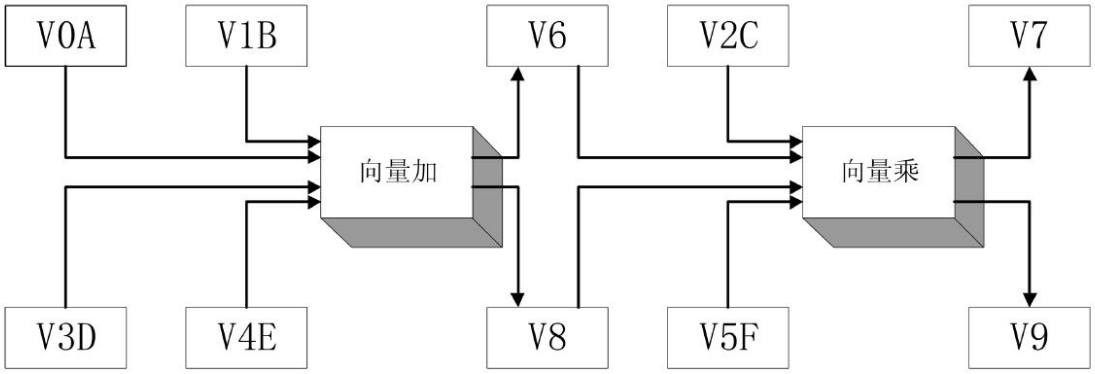
$T_{总共}$ = $T_{通过}$＋（64−1）＝23+63=86（拍）

3.19 某向量处理机有 16 个向量寄存器，其中 V_0~V_5 中分别放有向量 A、B、C、D、E、F，向量长度均为 8，向量各元素均为浮点数；处理部件采用两条单功能流水线，加法功能部件时间为 2 拍，乘法功能部件时间为 3 拍。采用类似于 CARY-1 的链接技术，先计算（A+B）*C，在流水线不停流的情况下，接着计算（D+E）*F。

（1） 求此链接流水线的通过时间？（设寄存器入、出各需 1 拍）

（2） 假如每拍时间为 50ns，完成这些计算并把结果存进相应寄存器，此处理部件的实际吞吐率为多少 MFLOPS？

解：（1）我们在这里假设 A＋B 的中间结果放在 V6 中，（A＋B）×C 地最后结果放在 V7 中，D＋E 地中间结果放在 V8 中，（D＋E）×F 的最后结果放在 V9 中。具体实现参考下图：



通过时间应该为前者（（A＋B）×C）通过的时间：

$T_{通过}=(1+2+1)+(1+3+1)=9$（拍）

（2）在做完（A＋B）×C 之后，作（C＋D）×E 就不需要通过时间了。

V6←A＋B

V7←V6×C

V8←D＋E

V9←V8×F

第一个元素需要T通过=9**拍，**后续n-1个元素每个一拍进入
同步+ (D+E)*F省去了通过时间 **只剩**n-1个元素进入

$$T = T_{通过} + (8-1) + 8 = 24(拍) = 1200(ns)$$

$$TP = \frac{32}{T} = 26.67MFLOPS$$

每次 2*(**加法**8+**乘法**8) = 32次浮点计算

在一台向量处理机上实现A＝B×s操作，其中A和B是长度为N=200的向量，s是一个标量。向量寄存器长度MVL=64。各功能部件的启动时间为：取数和存数部件为12个时钟周期、乘法部件为7个时钟周期，执行标量代码的开销T_{loop}=15个时钟周期，对一个向量元素执行一次操作的时间T_g=一个时钟周期。计算A的总执行时间。

解：

因为向量长度超过了向量寄存器的长度，所以要采取分段开采方法。

每次循环主要由下面三条向量指令组成：

```
LV        V1，  Rb          ；取向量B
MULTVS  V2，  V1，  Fs      ；向量和标量相乘
SV        Ra，  V2          ；存向量
```

假设A和B的分别放在R_a和R_b之中，s在F_s中。

三条指令之间存在有写读数据相关，因此必须把它们分成3个编队，T_{chime}= 3。

$$T_{200}=4\times(15+T_{start})+200\times3$$
$$=60+(4\times T_{start})+600$$
$$=660+(4\times T_{start})$$

其中：$T_{start}=12+7+12=31$，

因此，$T_{200}=660+4\times31=784$

- 例2 某向量流水机有三个向量访存部件，其中的两个用于向量Load，一个用于向量Store。三个向量访存部件可同时使用。该流水机的向量寄存器长度为64。若要进行向量运算Z = X + s*Y（s为标量，已保存于标量寄存器S1中），且向量长度N小于64，则下列指令段可完成上述运算：

```
LOADV     V1, M(X)      ;6拍
LOADV     V2, M(Y)      ;6拍
MULSV     V3, V2, S1;7拍
ADDV      V4, V1, V3 ;6拍
STOREV    V4, M(Z)      ;6拍
```

分别求出以上指令段采用串行方法（向量访存部件可并行工作）以及链接方法的执行时间（读写寄存器各需1拍），并画出链接操作示意图。

- 串行：共四个编队，时间为

(1+6+1+N-1)+ (1+7+1+N-1)+ (1+6+1+N-1)+ (1+6+1+N-1) =

- 链接： 共两个链接编队，时间为

(1+6+1+N-1)+ (1+7+1)+ (1+6+1+N-1)+ (1+6+1) =

```
LOADV    V1, M(X)      ;6拍
LOADV    V2, M(Y)      ;6拍
MULSV    V3, V2, S1;7拍
ADDV     V4, V1, V3    ;6拍
STOREV   V4, M(Z) ;6拍
```

- I1,I2,I3为一链接编队，I4,I5为一链接编队

$$\left\lceil \frac{n}{\mathrm{MVL}} \right\rceil \times (T_{\mathrm{start}} + T_{\mathrm{loop}}) + mn$$

第七章 多处理机TLP

线程级并行（TLP）

1. **定义**：利用多线程并发执行实现并行，基于 MIMD 模型，适用于紧耦合共享存储多处理机。
2. **实现方式**：多处理器、多核处理器、多线程处理器。
3. **与 ILP 对比**：单处理器通过指令级并行（ILP）优化，多处理器通过 TLP 提升整体性能。

1.集中式共享存储（SMP/UMA）
2.分布式共享存储（DSM/NUMA）

通信模型分类

1. **共享内存模型**

   通信方式：通过 Load/Store 指令隐式通信，适用于数据交互频繁场景。

2. **消息传递模型**

   通信方式：显式发送消息，如 MPI 接口，适合大规模并行计算。

缓存一致性与存储同一性

- 写协议
- **侦听协议（Snooping）**

  典型协议：MSI（Modified/Shared/Invalid）、MESI（增加 Exclusive 状态）、MOESI（增加 Owner 状态）。
- **目录协议（Directory）**

  原理：中央目录记录缓存块状态，适用于大规模分布式系统。

  优化方案：有限指针（LPS）、稀疏目录（Sparse Directory）减少存储开销。

缓存一致性

协议	状态数	适用场景	优势
MSI	3	小规模 SMP	实现简单
MESI	4	多核处理器	减少总线事务
MOESI	5	复杂多核系统	优化写回流程
目录协议	-	大规模 DSM	可扩展性强

缓存一致性与存储同一性

1. 规定多处理器访问内存操作的全局顺序，确保程序执行正确性。
2. **常见模型**
   1. **顺序一致性（Sequential Consistency）**：所有操作按程序序执行，结果等价于单处理器顺序执行。
   2. **弱一致性（Weak Ordering）**：仅在同步点保证顺序，提升性能。
   3. **释放一致性（Release Consistency）**：区分获取（ACQUIRE）和释放（RELEASE）操作，优化同步开销。

考核形式

（一）核心概念辨析

1. **TLP 与 ILP 的区别**：TLP 通过多线程并行，ILP 通过指令流水线并行。
2. **UMA 与 NUMA 的性能差异**：UMA 访问延迟一致但扩展性差，NUMA 支持大规模节点但需数据本地化。

（二）典型计算例题

1. **加速比计算**：应用 Amdahl 定律，如串行比例 20%，16 处理器的理论加速比 = 1/(0.2+0.8/16)=5。
2. **CPI 计算**：考虑远程访问开销，如 0.5% 远程访问，延迟 200 周期，CPI=1+0.5%×200=2。

（三）协议状态迁移

- 以 MESI 协议为例，掌握 Cache 块在 Modified、Exclusive、Shared、Invalid 状态间的迁移条件及总线事务。

其他为补充题，供自学

习题6.7（GCD测试方法）

用GCD 测试法判断下面的循环中是否存在循环携带的真数据相关。

```
for (i=2;i<=100;i+2)
    a[i]=a[i-1];
```

解：在使用GCD测试之前，必须先对这段代码进行“规范化”修改下标从1开始，而且每次循环后增加1。

For(i=1;i<=100;i+=2)　　a[i]=a[i-1];

规范化为：For(k=1;k<=50;k++)　　a[2K]=a[2K-1];

在这个循环中a=2，b=0，c=2，d=-1，

这样GCD(a,c)=2，　d-b=-1，

由于前者不能够整除后者；

该循环不存在循环携带的真数据相关。

习题6.8（循环展开）

表6.1本节使用的浮点流水线的延迟

产生结果的指令	使用结果的指令	延迟(cycles)
浮点计算	另一个浮点计算	3
浮点计算	浮点store(S.D)	2
浮点Load(L.D)	浮点计算	1
浮点Load(L.D)	浮点store(S.D)	0

整数运算，分支延迟和load需要一个周期延迟，

如果分支的寄存器在前一条指令计算出，也需要一个周期延迟，

因为整数计算在第3个周期完成，而分支第2个周期就用到

DADDIU R1, R1, #-8	7
(空转)	8
BNE R1, R2, Loop	9

习题6.8

在不进行指令调度的情况下，程序的实际执行情况如下：

		指令流出时钟
Loop:	L.D F0, 0(R1)	1
	L.D F4, 0(R2)	2
	(空转)	3
	MUI.D F0, F0, F4	4
	(空转)	5
	(空转)	6
	(空转)	7
	ADD.D F2, F0, F2	8
	DADDIU R1, R1, #-8	9
	DADDIU R2, R2, #-8	10
	BNE R1, R3, Loop	11
	(空转)	12

计算原程序周期数：每对元素所需的时钟周期数=12，其中空转数=5；

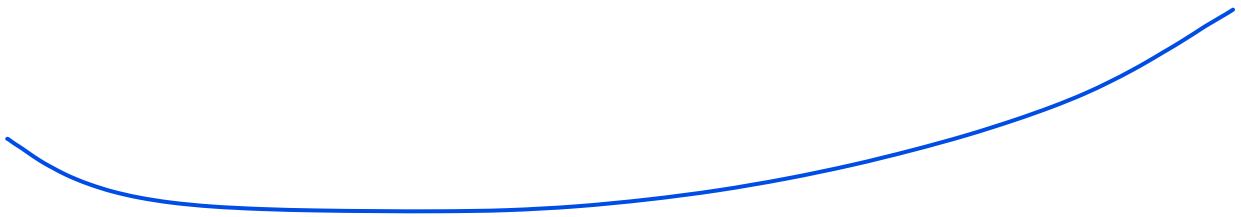
习题6.8 新程序

```
Loop:   L.D  F0,16(R1)        ;F0 ←A(i+2)
        L.D  F4,16(R2)        ;F4 ←B(i+2)
        L.D  F6,8(R1)         ;F6 ←A(i+1)
        MUL.D  F0,F0,F4       ;F0 ←A(i+2) ×B(i+2)
        L.D  F8,8(R2)         ;F8←B(i+1)
        L.D  F10,0(R1)        ;F10 ←A(i)
        MUL.D  F6,F6,F8       ;F6 ←A(i+1) ×B(i+1)
        ADD.D  F2,F0,F2       ; F2 ←F2+ A(i+2) ×B(i+2)
        L.D  F12,0(R2)        ;F12←B(i)
        DADDUI  R1,R1,-24     ;R1 ←R1-24
        MUL.D  F10,F10,F12    ;F10 ←A(i) ×B(i)
        ADD.D  F2,F6,F2       ; F2 ←F2+ A(i+1) ×B(i+1)
        DADDUI  R2,R2,-24     ;R2 ←R2-24
        BNE   R1,R3,loop      ;若R1 ≠R3,循环
        (空转)
        ADD.D  F2,F10,F2      ; F2 ←F2+ A(i) ×B(i)
```

新程序周期数：每对元素所需的时钟周期数=16/3=5.3，其中空转数=1/3=0.3

习题6.8 新程序

```
L.D F0, 0(R1)
L.D F6,-8(R1)
L.D F4, 0(R2)
L.D F8,-8(R2)
MUL.D F0, F0, F4; 1 from L.D F4, 0(R2)
MUL.D F6, F6, F8; 1 from L.D F8,-8(R2)
DADDIU R1, R1,-16
DADDIU R2, R2,-16
ADD.D F2, F0, F2; 3 from MUL.D F0, F0, F4
BNEZ R1, foo
ADD.D F10, F6, F10; 3 from MUL.D F6, F6, F8and fill br anch del ay sl ot
bar:
ADD.D F2, F2, F10; combine even and odd elements
```



习题8.11（I/O与Cache数据一致性）

假设在一个计算机系统中：

（1）每页为32KB，Cache块大小为128B

（2）对应新页的地址不在Cache中，CPU不访问新页中的任何数据；

（3）Cache中95%的被替换块将再次被读取，引起一次不命中；

（4）Cache使用写回方法，平均60%的块被修改过；

（5）I/O系统缓冲能够存储一个完整的cache块

（6）访问或不命中在所有cache块中均匀分布；

（7）在CPU和I/O之间，没有其他访问cache的干扰；

（8）无I/O时，每100万个时钟周期内有18000次不命中；

（9）不命中开销是40个时钟周期，如果被替换的块被修改过，则再加上30个周期用于写回主存；

（10）假设计算机平均每200万个周期处理一页

试分析I/O对于性能的影响有多大

习题8.11

解：每个主存页有32K/128＝256块。

按块传输，I/O传输本身并不引起Cache失效。但可能要替换Cache中的有效块。如果替换块中有60%被修改过，将需要256×60%×30＝4608个时钟周期将这些脏块写回主存。

替换出去的块中，有95%的再次被访问，从而产生95%×256＝244次失效，将再次发生替换。由于这次被替换的244块中数据是从I/O直接写入Cache的，因此所有块都为被修改块，需要写回主存（因为CPU不会直接访问从I/O来的新数据），需要244×（40＋30）＝17080个时钟周期。

写直达

没有I/O时，每一页平均使用200万个时钟周期，Cache失效36000次，其中60%被修改过，所需的IO时间为：

=36000×40＋36000×60%×30＝2088000（时钟周期）

时钟I/O造成的额外性能损失比例为

（4608＋17080）÷（2000000＋2088000）＝0.53%

各种互连函数总结

交换置换函数定义： $E(X_{n-1}X_{n-2}\cdots X_1X_0)=X_{n-1}X_{n-2}\cdots X_1X_0$， 其中$0\leqslant i\leqslant n-1$

立方体函数定义：Cubei的功能是对入端结点编号二进制形式的第i位取反

$Cube_i(X_{n-1}\cdots X_{i+1}X_iX_{i-1}\cdots X_0)=X_{n-1}\cdots X_{i+1}\overline{X_i}X_{i-1}\cdots X_0$， 其中$0\leqslant i\leqslant n-1$

均匀洗牌 σ ： shuffle（$X_{n-1}X_{n-2}\cdots\cdots X_0$）= $X_{n-2}\cdots\cdots X_0X_{n-1}$（循环左移）

蝶式互连函数butterfly β：最高位与最低位互换；

反位序函数 ρ：就是二进制各位次序颠倒过来；

PM2I函数定义：$PM2_{\pm i}$的功能是对入端结点编号加或减2^i，然后再作模N运算

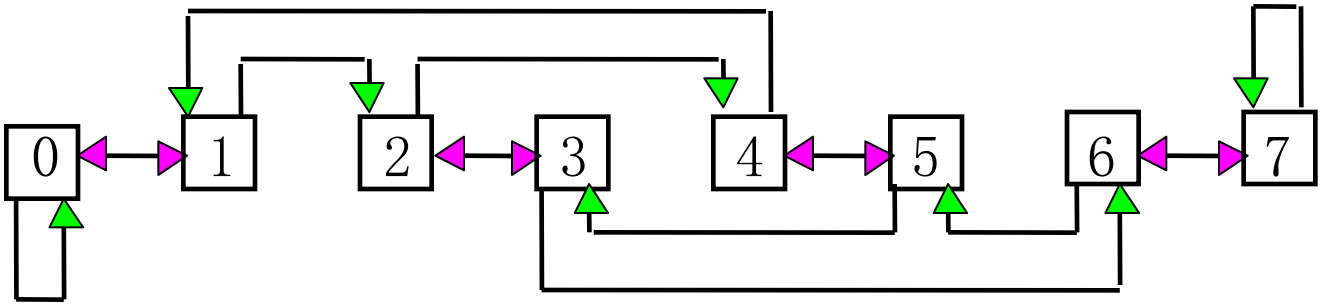
$$PM2_{+i}(X) = X + 2^i \mod N$$

$$PM2_{-i}(X) = X - 2^i \mod N$$

其中 X = 0 ～ N - 1，i = 0 ～ n - 1。

几个单级静态网络参数

- 单级混洗—交换网络
  - 网络的直径是2n-1。
  - 0和N-1的入度和出度各为1，其它的各为2；



- 单级PM2I网络
  - 2n-1种不同的置换，度为2n-1；
  - 单级PM2I网络的直径是 $\lceil n/2 \rceil$
- n-立方体中结点的度都是n，直径也是n

习题9.9

$$Cube_2(12) = Cube_2(01100B) = 01000B = 8$$

$$\sigma(8) = \sigma(01000B) = 10000B = 16$$

$$\beta(9) = \beta(01001B) = 11000B = 24$$

$$PM2I_{+3}(28) = PM2I_{+3}(11100B) = 11100B + 01000B \bmod 2^5 = 00100B = 4$$

$$Cube_0(\sigma(4)) = Cube_0(\sigma(00100B)) = 01001B = 9$$

$$\sigma(Cube_0(18)) = \sigma(Cube_0(10010B)) = 00111B = 7$$

(2)2^5个结点的混洗交换网的直径是2n-1 =2×5-1=9;

从5号处理机（00101B）发送数据到7号处理机（00111B），最短路径要经过6步，包含5步左移和1步求反（因为00101BXOR00111B=00010B），经过的处理机编号为：

00101B→01010B →10100B →01001B →10010B →10011B →00111B

（3）网络直径是「5/2」=3;

结点度是2n-1 =2×5-1=9;

与2号处理机距离最远的是13、15、21、23号处理机。

习题9.13（多级互连网络）

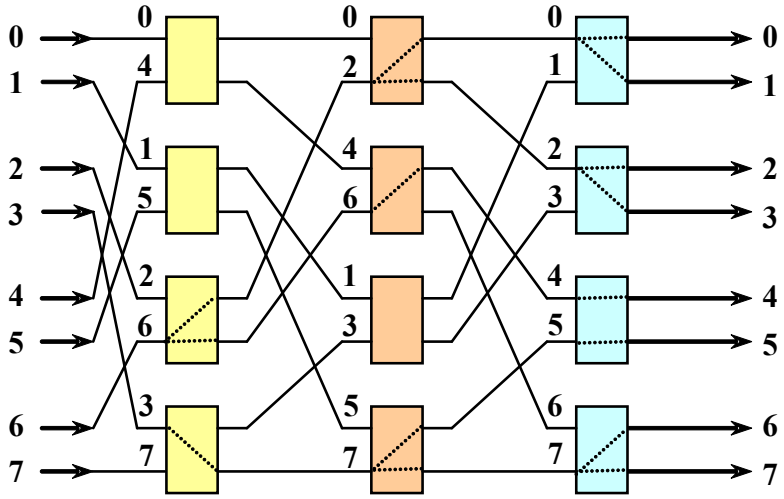
有$\log_2 N$级，每级用$N/2$个2×2开关，共需要$N/2*\log_2 N$个开关。

用N=8的三级Omega网络连接8个处理机（P0～P7），

8个处理机的输出端分别依次连接Omega的8个输入端0～7， 8个处理机的输入端分别依次连接Omega的8个输出端0～7，处理机P6要把数据播送到处理机P0～P4，

处理机P3要把数据播送到处理机P5～P7，

能否同时为它们的播送要求实现连接，画出开关状态图。



习题10.6（并行处理对性能的提高）

一个具有32台处理机的系统，对远程存储器访问时间是2000ns。除了通信以外，假设计算中的访问均命中局部存储器。当发出一个远程请求时，本地处理机挂起。处理机的时钟周期是10ns，假设指令基本的CPI为1.0（假设所有访存均命中cache）。对于下述两种情况：

（1）没有远程访问；

（2）0.5%的指令需要远程访问.

试问前者比后者快多少？

解：已知远程访问率 p = 0.5%，远程访问时间 t = 2000ns，时钟周期 T = 10ns

远程访问开销 C = t/T = 2000ns/10ns = 200（时钟周期数）

有0.5%远程访问的机器的实际CPI2为：CPI2 = CPI1+p×C =1.0+0.5%×200 = 2.0

只有局部访问的机器的基本 CPI1 = 1.0，CPI2/ CPI1 = 2.0/1.0 = 2（倍）

因此，没有远程访问状态下的机器速度是有0.5% 远程访问的机器速度的2倍。

习题10.9（栅栏同步，旋转锁）

采用排队锁和fetch-and-increment重新实现栅栏同步，并将分别与采用旋转锁实现的栅栏同步进行性能对比。

解：

```
fetch-and-increment(count);
if(count==total){                    //进程全部到达
      count = 0;                     //重置计数器
      release=1;                     //释放进程
}else{                               //还有进程未到达
      spin(release=1);  //等待信号
}
```

当有N个处理器时，上述代码执行fetch-and-increment 操作N 次

当访问释放操作的时候，有N个Cache未命中

当最后一个处理器到达栅栏条件后，release被置为“1”，此时有N-1个Cache未命中（对于最后一个到达栅栏的处理器，当它读release 的时候，将在主存中命中）

所以，共有3N-1次总线传输操作。如果有10个处理器，则共有29 次总线传输操作，总共需要2900个时钟周期。

旋转锁实现的栅栏同步性能

```
lock（counterlock）;
count=count+1;
unlock（counterlock）;
if（count= =total）{
count=0;
release=1;
}
else {
spin（release=1）;
}
```

第i个处理器通过栅栏产生的事件序列

事件	数量	对应源代码	说明
LL	i	Lock(counterlock);	所有处理器抢锁
SC	i	Lock(counterlock);	一个成功
LD	1	count=count+1;	读count
SD	1	count=count+1;	写count
LL	i-1	Lock(counterlock);	不成功的处理器再次抢锁
SD	1	unlock(counterlock);	获得锁产生的失效
LD	2	spin(release=1);	读release：初次和最后写产生的失效，最后一个处理器只需要1次写操作。

n 个进程通过旋转锁栅栏产生的总线事务总数：

$$\sum_{i=1}^{n}(3i+4)-1=(3n^2+11n)/2-1$$

代入条件n=10，得总线事务数为204个，折合20400个时钟周期。

习题10.9

排队锁实现的栅栏同步性能

栅栏采用排队锁实现--在教材P323程序基础上，加算其它指令产生的总线事务

Lock	1（成功） 2（不成功）
Count	2
Unlock	1
N（else）	2
Y	2（count=0）+release（1）

```
lock（counterlock）;
count=count+1;
unlock（counterlock）;
if（count= =total）{
count=0;
release=1;
}
else {
spin（release=1）;
}
```

n 个进程通过排队锁栅栏产生的总线事务总数6+7（n-2）+7

代入条件n=10，得总线事务数为70个，折合6999个时钟周期。